L14 SVM Math and Numerical

Find the hyperplane

- Problem statement 1 : Construct a simple SVM classifier that separates the two classes
- Positively labelled data points (3,1)(3,-1)(6,1)(6,-1) and
- Negatively labelled data points (1,0)(0,1)(0,-1)(-1,0)
- For all negative labelled output is -1 and for all positive labelled output is 1
- Draw the hyperplane

- Ref: SVM Example by Dan Ventura, 2009
- https://axon.cs.byu.edu/Dan/678/miscellaneous/SVM.example.pdf

Find the hyperplane

- Steps

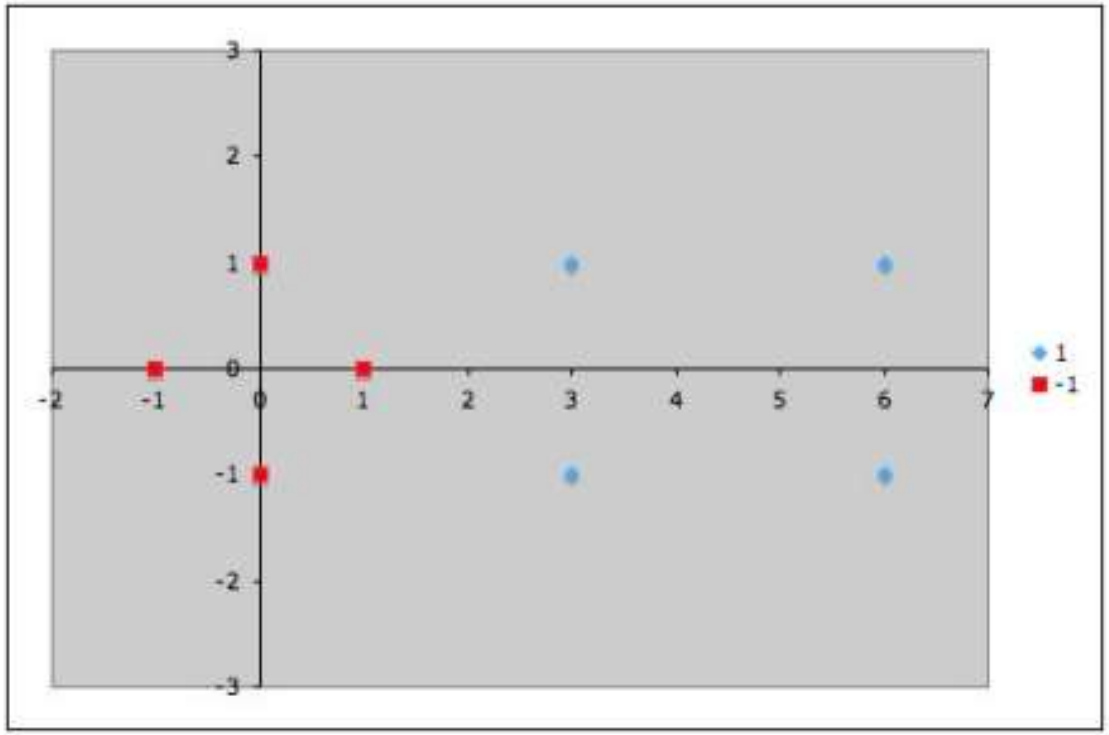
1. Plot the graph and identify whether the data is linearly separable. If so, use linear SVM
2. Identify the support vectors or nearest data points by inspection on the graph
3. Construct augmented vectors by adding the bias input and solve the system of linear equations for αi
4. Compute the weight vector, wi= Σ αi * si
5. Draw the hyperplane

Find the hyperplane

Step 1: Plot the graph and identify whether the data is linearly separable. If so, use linear SVM

Positively labelled data points (3,1)(3,-1)(6,1)(6,-1) and

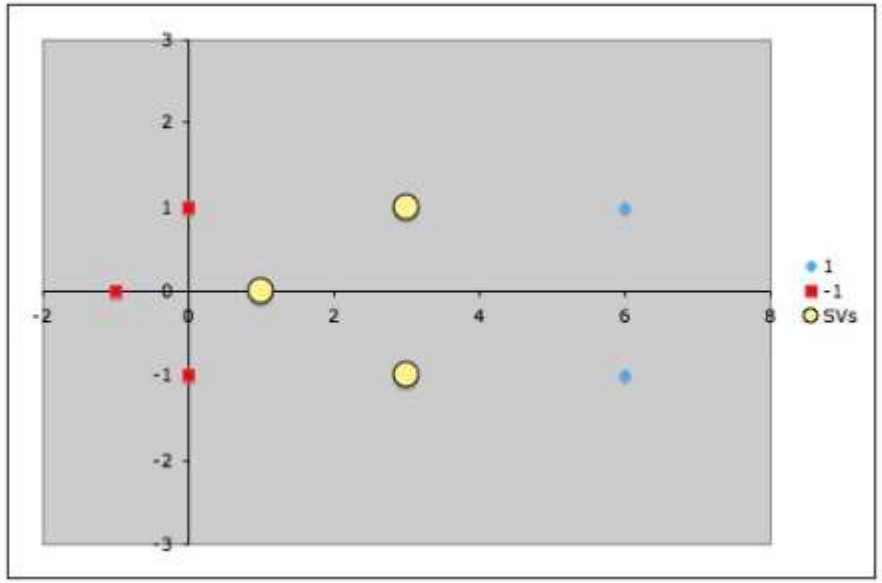
Negatively labelled data points (1,0)(0,1)(0,-1)(-1,0)



Blue diamonds are positive examples and red squares are negative examples

Find the hyperplane

- Step 2: Identify the support vectors or nearest data points by inspection on the graph
- Since the data is linearly separable, we can use a linear SVM
- By inspection, there are three support vectors
- The three support vectors are marked as yellow circles



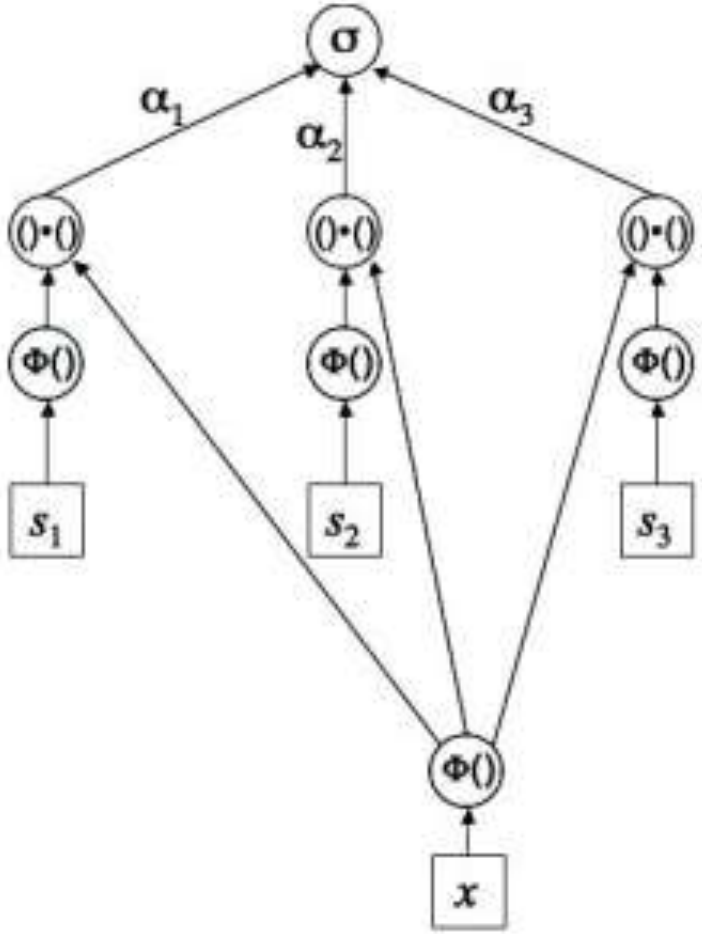
Find the hyperplane

- Step 3: Construct augmented vectors by adding the bias input and solve the system of linear equations for αi
- Use vectors augmented with a 1 as a bias input
- For clarity differentiate with an over-tilde.
- So, if s1 = (1 0), then s˜1 = (1 0 1).
- mapping function $\Phi()$ is the identity function
- Find values for the αi such that
- $\alpha 1\Phi(s1) \cdot \Phi(s1) + \alpha 2\Phi(s2) \cdot \Phi(s1) + \alpha 3\Phi(s3) \cdot \Phi(s1) = -1$
- $\alpha 1\Phi(s1) \cdot \Phi(s2) + \alpha 2\Phi(s2) \cdot \Phi(s2) + \alpha 3\Phi(s3) \cdot \Phi(s2) = +1$
- $\alpha 1\Phi(s1) \cdot \Phi(s3) + \alpha 2\Phi(s2) \cdot \Phi(s3) + \alpha 3\Phi(s3) \cdot \Phi(s3) = +1$
- let $\Phi() = I$. So,
- $\alpha 1\tilde{s}1 \cdot \tilde{s}1 + \alpha 2\tilde{s}2 \cdot \tilde{s}1 + \alpha 3\tilde{s}3 \cdot \tilde{s}1 = -1$ — $\tilde{s}1$ is common term in Eqn 1
- $\alpha 1\tilde{s}1 \cdot \tilde{s}2 + \alpha 2\tilde{s}2 \cdot \tilde{s}2 + \alpha 3\tilde{s}3 \cdot \tilde{s}2 = +1$ — $\tilde{s}2$ is common term in Eqn 2
- $\alpha 1\tilde{s}1 \cdot \tilde{s}3 + \alpha 2\tilde{s}2 \cdot \tilde{s}3 + \alpha 3\tilde{s}3 \cdot \tilde{s}3 = +1$ — $\tilde{s}3$ is common term in Eqn 3

Find the hyperplane

- $\alpha 1 \tilde{s}1 \cdot \tilde{s}1 + \alpha 2 \tilde{s}2 \cdot \tilde{s}1 + \alpha 3 \tilde{s}3 \cdot \tilde{s}1 = -1$
- $\alpha 1 \tilde{s}1 \cdot \tilde{s}2 + \alpha 2 \tilde{s}2 \cdot \tilde{s}2 + \alpha 3 \tilde{s}3 \cdot \tilde{s}2 = +1$
- $\alpha 1 \tilde{s}1 \cdot \tilde{s}3 + \alpha 2 \tilde{s}2 \cdot \tilde{s}3 + \alpha 3 \tilde{s}3 \cdot \tilde{s}3 = +1$
- Compute the dot products
- $2\alpha 1 + 4\alpha 2 + 4\alpha 3 = -1$
- $4\alpha 1 + 11\alpha 2 + 9\alpha 3 = +1$
- $4\alpha 1 + 9\alpha 2 + 11\alpha 3 = +1$

The SVM architecture



Find the hyperplane

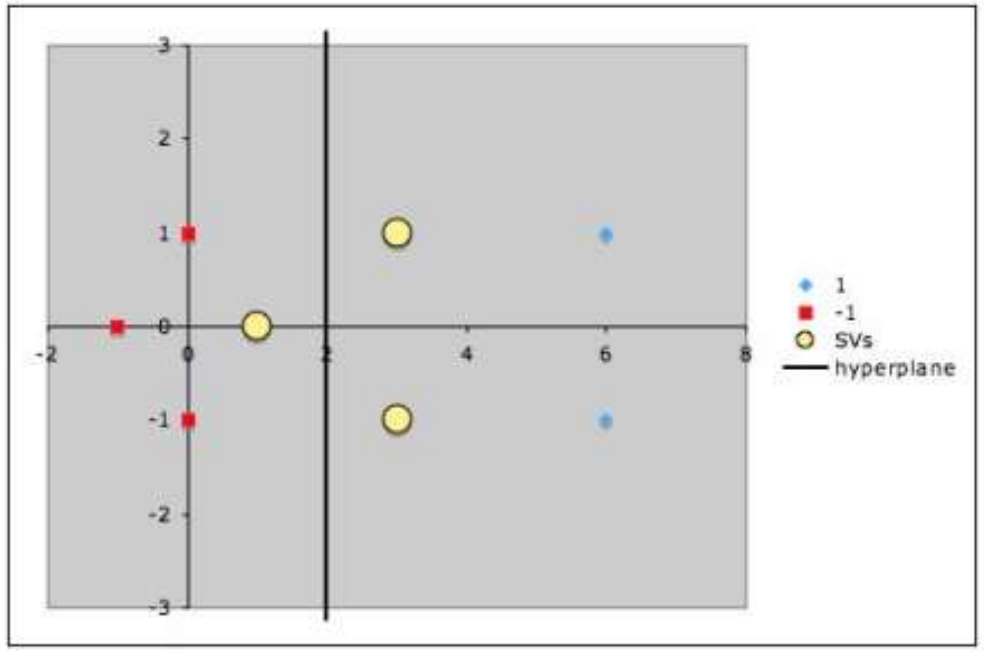
- Step 4: Compute the weight vector, $w_i = \Sigma \alpha_i * s_i$
- Solve the system of linear equations
- $\alpha 1 = -3.5$, $\alpha 2 = 0.75$ and $\alpha 3 = 0.75$.
- How these α values relate to the hyperplane?
- We have the αi, , how do we find the hyperplane that discriminates the positive from the negative example
- $\tilde{w} = \Sigma_i \alpha_i * \tilde{s}_i$
- $-3.5 (1\ 0\ 1) + 0.75 (3\ 1\ 1) + 0.75 (3\ -1\ 1) = (1\ 0\ -2)$
- Note that vectors are augmented with a bias, we can equate the last entry in ˜w as the hyperplane offset b which is -2
- write the separating hyperplane equation y = wx+b with w = (1 0) and b = −2.
- Plotting the line gives the expected decision surface

Find the hyperplane

Step 5: Draw the hyperplane

The discriminating hyperplane corresponding to the values $\alpha 1 = -3.5$, $\alpha 2 = 0.75$ and $\alpha 3 = 0.75$

The separating hyperplane equation $y = wx+b$ with $w = (1\ 0)$ and $b = -2$



Implementation of Problem Statement 1

Implementation of Problem Statement 1

```
# Import necessary libraries
import numpy as np
from sklearn import svm
import matplotlib.pyplot as plt
# Define the training data and labels
X = np.array([[3, 1], [3,-1], [6, 1], [6,-1],[1, 0], [0,1], [0,-1], [-1,0]])
y = np.array([1, 1, 1, 1, 0, 0, 0, 0])
# Create an SVM classifier with a linear kernel
clf = svm.SVC(kernel='linear’)
# Train the SVM classifier on the training data
clf.fit(X, y)
# Define a new data point for prediction
new_data_point = np.array([[4, 3]])
```

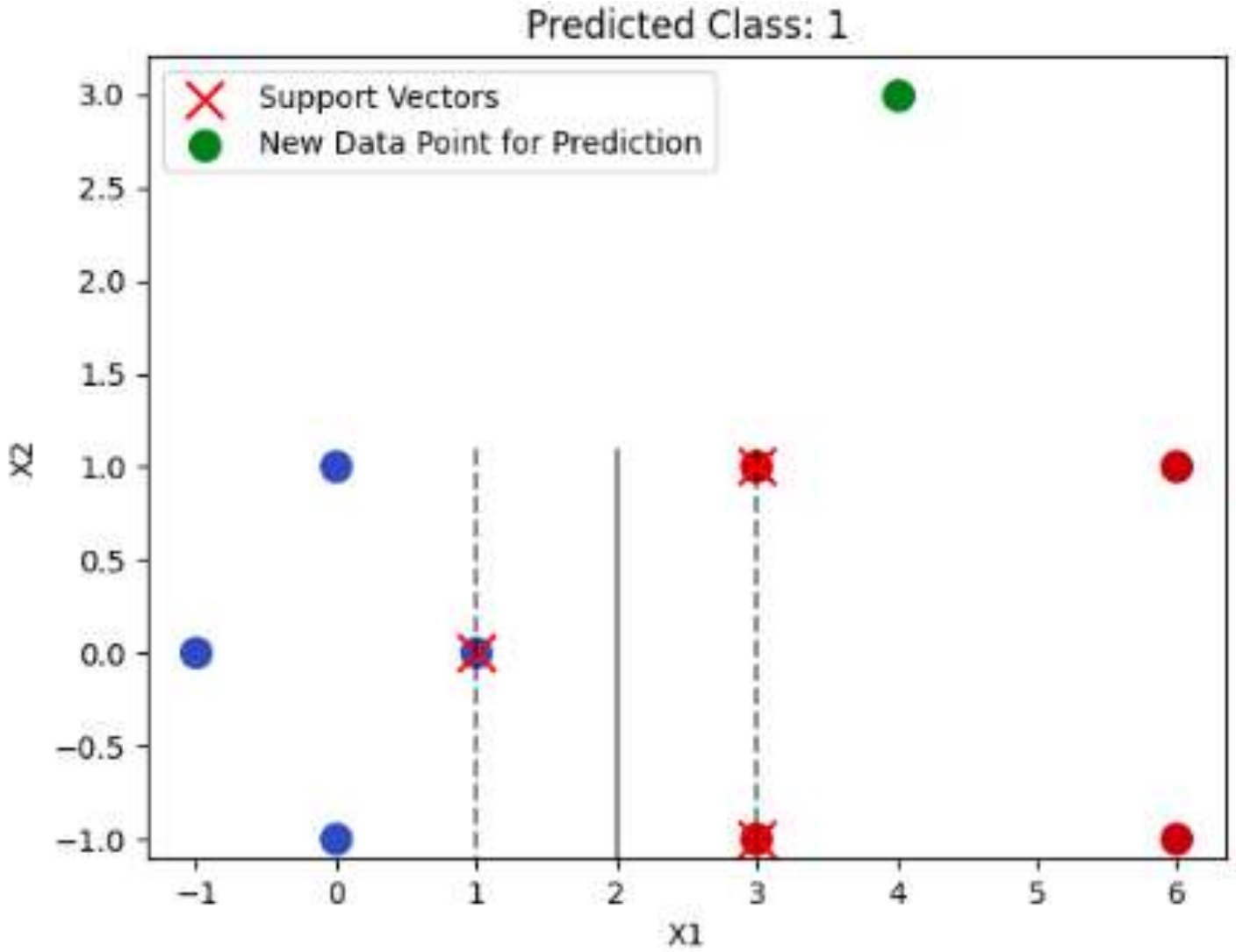
Implementation of Problem Statement 1

```
# Predict the class of the new data point using the trained SVM classifier
predicted_class = clf.predict(new_data_point)
# Get the support vectors
support_vectors = clf.support_vectors_
# Get the coefficients of the hyperplane (weights and bias)
w = clf.coef_[0]
b = clf.intercept_[0]
# Plot the training data points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', s=100)
# Plot the support vectors
plt.scatter(support_vectors[:, 0], support_vectors[:, 1], c='r', marker='x', s=150,
label='Support Vectors')
```

Implementation of Problem Statement 1

```
# Plot the decision boundary (hyperplane)
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 50), np.linspace(ylim[0], ylim[1], 50))
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--’])
# Plot the new data point for prediction
plt.scatter(new_data_point[0, 0], new_data_point[0, 1], c='g', marker='o', s=100, label='New Data Point for Prediction’)
# Set labels and legend
plt.xlabel('X1’)
plt.ylabel('X2’)
plt.legend(loc='best’)
# Show the plot
plt.title(f'Predicted Class: {predicted_class[0]}’)
plt.show()
```

Implementation of Problem Statement 1- Output



Problem statement 2 : Construct a simple SVM classifier that separates the two classes

Find the hyperplane

- Problem statement 2 : Construct a simple SVM classifier that separates the two classes
- Positively labelled data points (4,1), (4,-1), (6,0) and
- Negatively labelled data points (1,0), (0,1), (0,-1)
- For all negative labelled output is -1 and for all positive labelled output is 1
- Draw the hyperplane

Find the hyperplane

- Steps

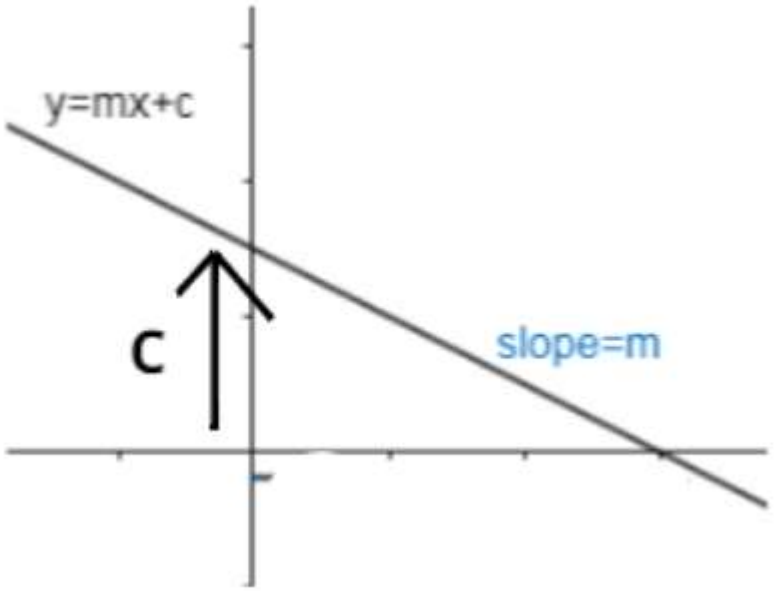
1. Plot the graph and identify whether the data is linearly separable. If so, use linear SVM
2. Identify the support vectors or nearest data points by inspection on the graph
3. Construct augmented vectors by adding the bias input and solve the system of linear equations for αi
4. Compute the weight vector, wi= Σ αi * si
5. Draw the hyperplane

Computing the margin

Find the best hyperplane between two hyperplanes

- Given two hyperplanes for SVM classifier 1 and SVM Classifier 2, find the best hyperplane?
- Distance between separating hyperplanes and support vector is margin.
- Thus, the best hyperplane will be the one whose margin is the maximum
- How to find the margin?
- Background on Vectors

Equation of hyperplane



Equation of a Hyperplane in 2 dimensions

$$\beta_0 + \beta_1 \mathbf{X_1} + \beta_2 \mathbf{X_2} = 0$$

Equation of a Hyperplane in p-dimensions

$$\beta_0 + \beta_1 \mathbf{X_1} + \ldots + \beta_p \mathbf{X_p} = 0$$

We have points on the hyperplane:

$$\mathbf{X} = [\mathbf{X_1}, \ldots, \mathbf{X_p}]^T$$

where $\mathbf{X}$ is a feature vector

In a 2-D space, we fit a line, Equation for a straight line with slope m and intercept c is $mx + c = 0$
The hyperplane equation dividing the points (for classifying) can be written as:

H: $\mathbf{w}^T\mathbf{x} + b = 0$
Here: b = Intercept or bias term of the hyperplane equation

Understanding the equation of the hyperplane

- An equation of a line is : $y=ax+b$
- However the equation of an hyperplane is defined by : $\mathbf{w}^T\mathbf{x}=0$
- How does these two forms relate ?
- In the hyperplane equation the name of the variables are in bold means that they are vectors.
- Moreover, w^Tx is how we compute the inner product or the dot product of two vectors
- A vector has magnitude (size) and direction which works perfectly well in 3 or more dimensions
- Therefore the application of “vector” is used in the Support Vector Machine algorithm
- Note that $y=ax+b$ is the same thing as $y-ax-b=0$

Understanding the equation of the hyperplane

Given two vectors $\mathbf{w}\begin{pmatrix} -b \\ -a \\ 1 \end{pmatrix}$ and $\mathbf{x}\begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$

$$\mathbf{w}^T\mathbf{x} = -b \times (1) + (-a) \times x + 1 \times y$$

$$\mathbf{w}^T\mathbf{x} = y - ax - b$$

- The two equations $\mathbf{w}^T\mathbf{x}$ and $y-ax-b$ are two different ways of expressing the same thing.
- It is interesting to note that w0 is −b, which means that this value determines the intersection of the line with the vertical axis.
- Why use the hyperplane equation $\mathbf{w}^T\mathbf{x}$ instead of $y=ax+b$?

Understanding the equation of the hyperplane

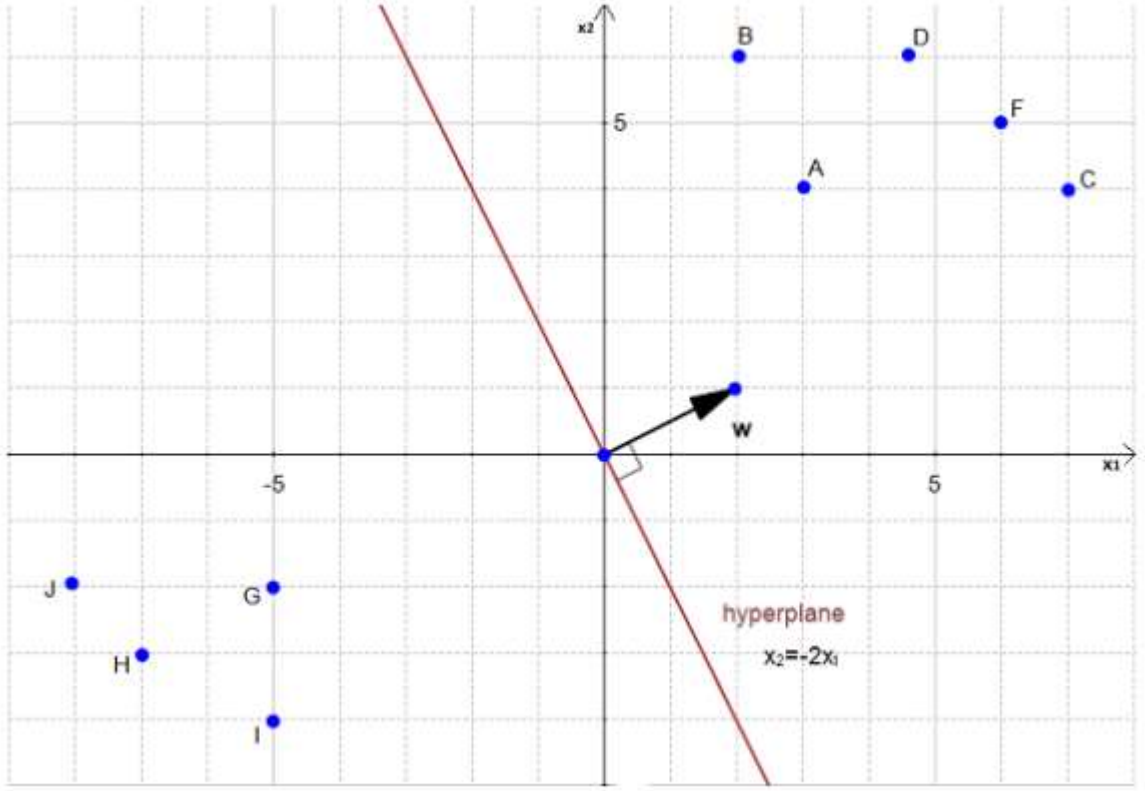
- Why use the hyperplane equation $\mathbf{w}^T\mathbf{x}$ instead of $y=ax+b$?
- For two reasons:

1. It is easier to work in more than two dimensions with this notation
2. The vector w will always be normal to the hyperplane.
    - w will always be normal because we use this vector to define the hyperplane, so by definition it will be normal.
    - when we define a hyperplane, we suppose that we have a vector that is orthogonal to the hyperplane

- Last property is used to compute the distance from a point to the hyperplane.

Compute the distance from a point to the hyperplane

- Consider the hyperplane which separates two group of data
- Verify the equation of hyperplane is x2=−2* x1



Compute the distance from a point to the hyperplane

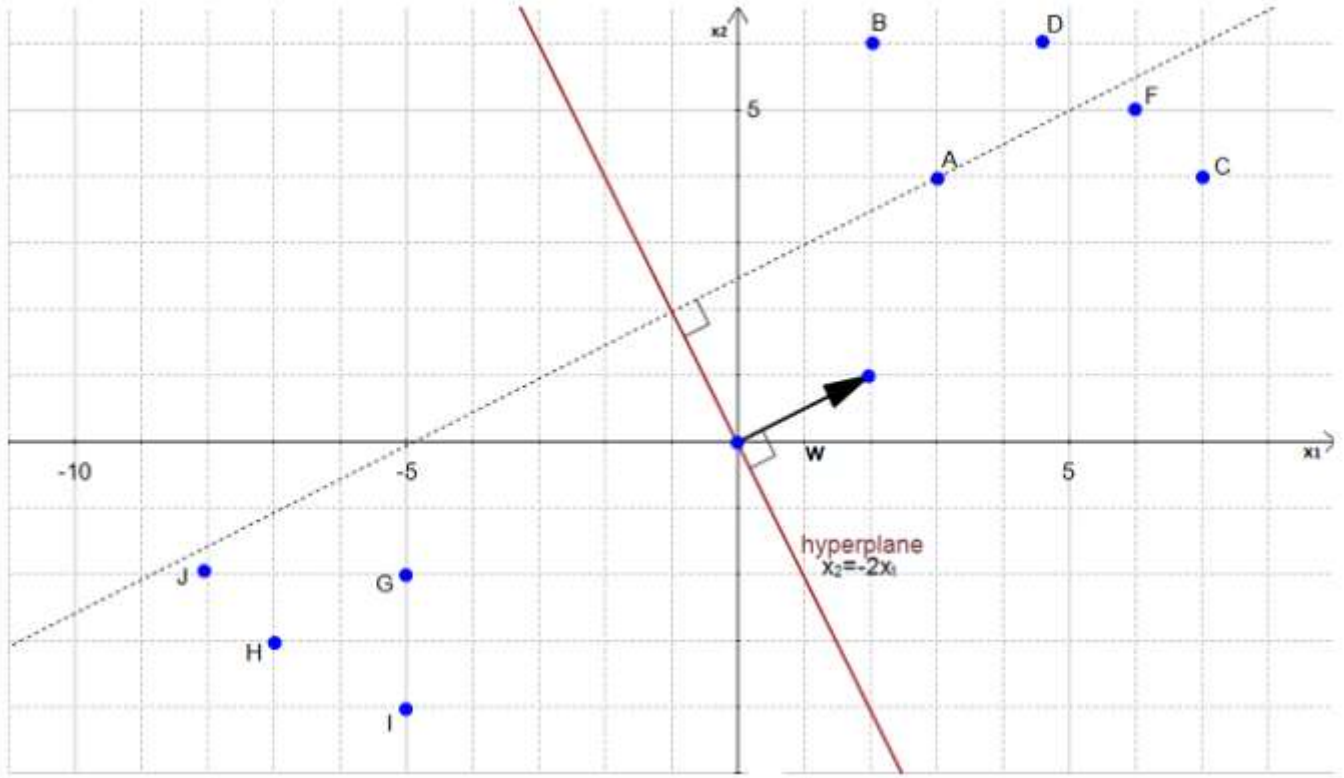
- To simplify the example, set w0=0
- The equation of the hyperplane x2=−2* x1 is same as x2+2x1=0
- This is equivalent to $\mathbf{w}^T\mathbf{x}$ =0

with $\mathbf{w}\begin{pmatrix}2\\1\end{pmatrix}$ and $\mathbf{x}\begin{pmatrix}x_1\\x_2\end{pmatrix}$

- Note that the vector w is not a data point
- We would like to compute the distance between the point A(3,4) and the hyperplane
- This is the distance between A and its projection onto the hyperplane

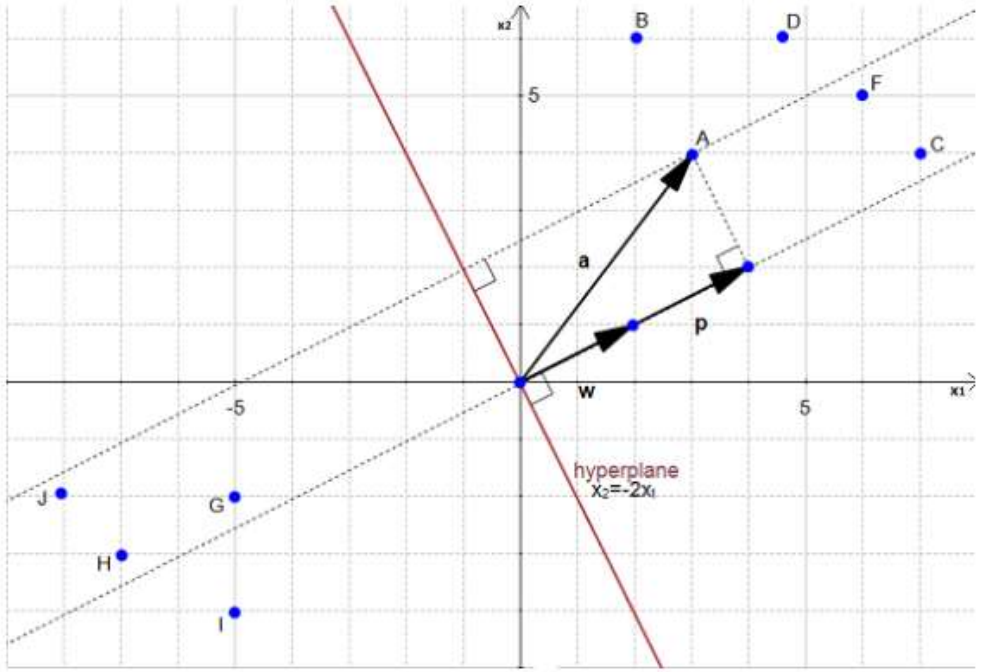
Compute the distance from a point to the hyperplane

Compute the distance between the point A(3,4) and the hyperplane



Compute the distance from a point to the hyperplane

- w=(2,1) is the vector normal to the hyperplane
- a=(3,4) is the vector between the origin and A
- We can view the point A as a vector from the origin to A
- If we project it onto the normal vector w, we get the vector p



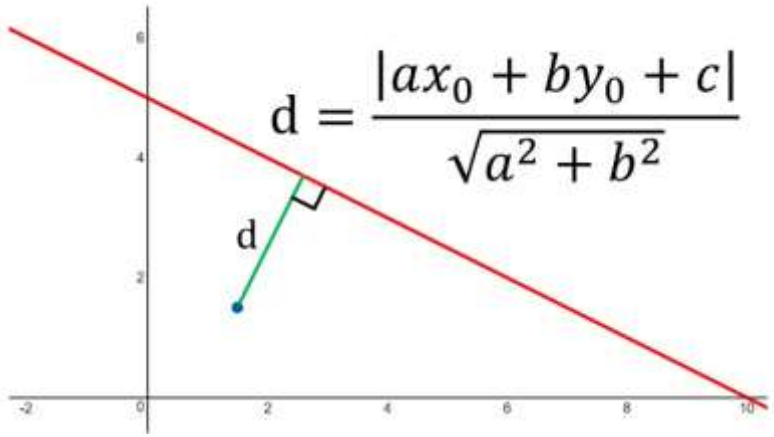
Compute the distance from a point to the hyperplane

- Compute the distance between the point A(3,4) and the hyperplane
- This distance is the same thing as ‖p‖
- Solve
- Ref: https://www.svm-tutorial.com/2014/11/svm-understanding-math-part-2/
- Published as eBook - Support Vector Machines succinctly by Alexandre Kowalczyk

Distance Measure

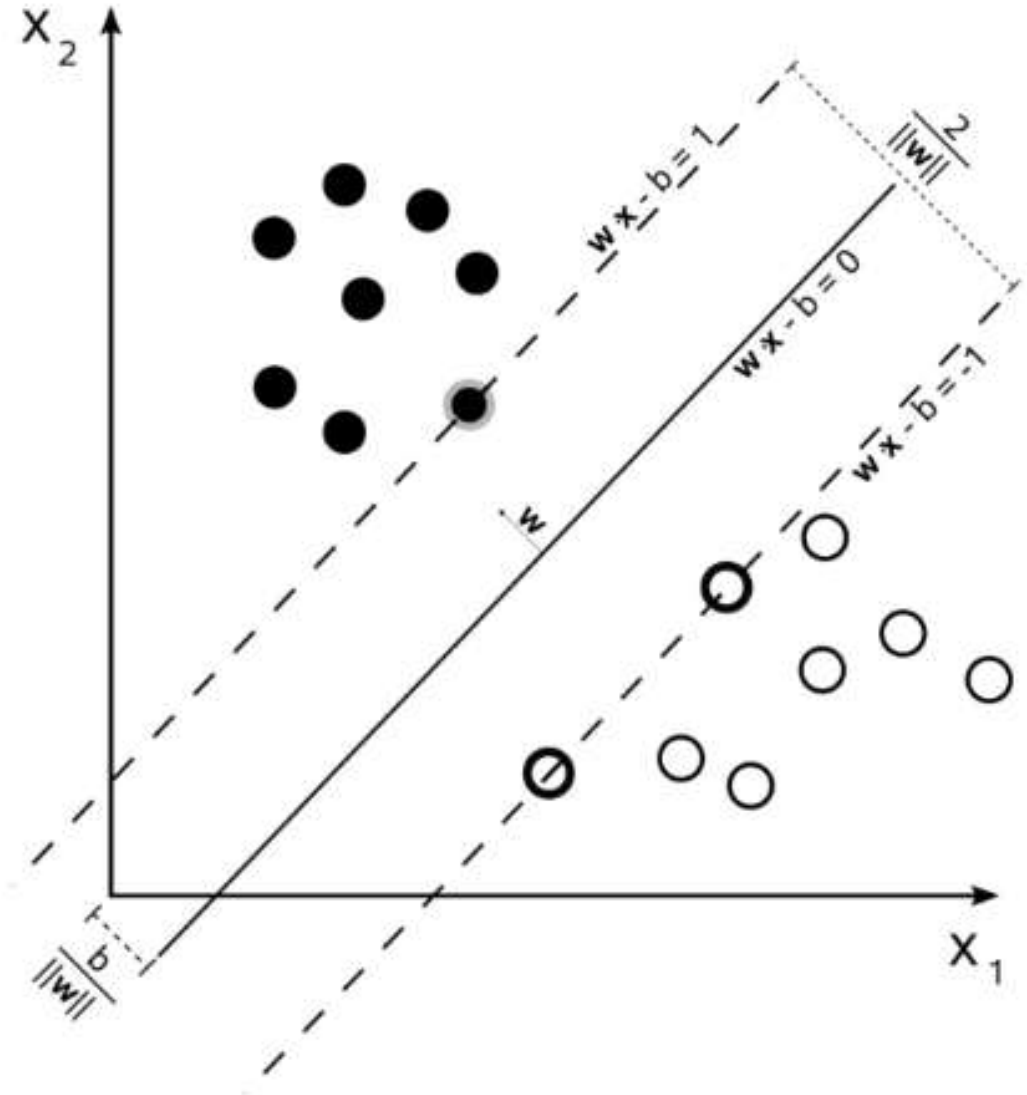
- The distance between a data point and the separating line
- The distance of any line, ax + by + c = 0 from a given point say, (x0 , y0) is given by d
- Similarly, the distance of a hyperplane equation: wTΦ(x) + b = 0 from a given point vector Φ(x0) can be easily written

- here ||w|| is the Euclidean norm for the length of w = $\sqrt{w_1^2 + w_2^2 + w_3^2 + \ldots w_n^2}$



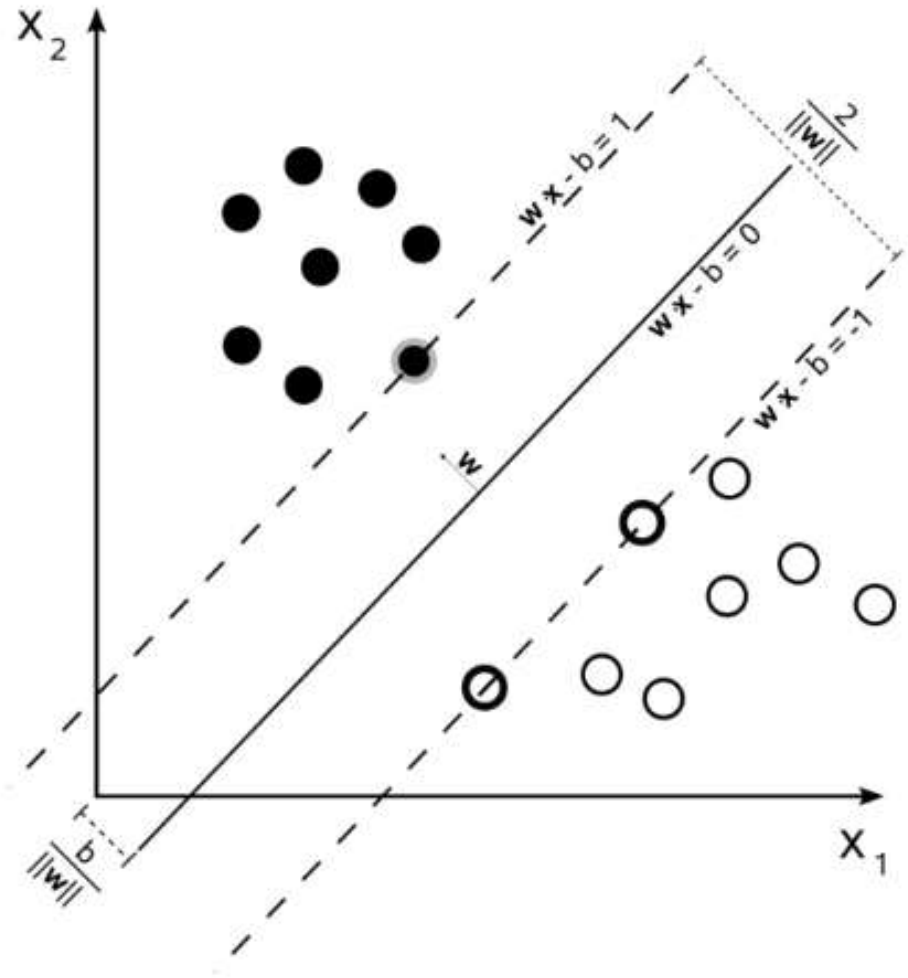
The distance from a point⟦(x⟧_0,y_0) to a line: Ax+By+c = 0 is: |Ax_0 + By_0 +c|/sqrt(A2+B2)

Distance Measure – how to find the geometric margin



Distance Measure – how to find the geometric margin

- W is normal to the hyperplane
- |b| / ||w|| is the perpendicular distance from the hyperplane to the origin
- ||w|| is the Euclidean norm of w
- w is a weight vector
- x is input vector
- b is bias



Distance Measure

- Measure the distance between hyperplanes wx−b=−1 and wx−b=1
- Let x0 be a point in the hyperplane wx−b=−1. So wx0−b=−1
- To measure the distance between hyperplanes wx−b=−1 and wx−b=1, we need to compute the perpendicular distance from x0 to plane wx−b=1 denoted as r
- Note that w/‖w‖ is a unit normal vector of the hyperplane wx−b=1.
- So (x0+ r w /‖w‖) should be a point in hyperplane wx−b=1 according to definition of r
- So, we have w(x0+r w /‖w‖)−b=1. Expanding this equation

$$
\begin{aligned}
& \mathbf{w}\mathbf{x}_0 + r\frac{\mathbf{w}\mathbf{w}}{\|\mathbf{w}\|} - b = 1 \\
\Longrightarrow\ & \mathbf{w}\mathbf{x}_0 + r\frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} - b = 1 \\
\Longrightarrow\ & \mathbf{w}\mathbf{x}_0 + r\|\mathbf{w}\| - b = 1 \\
\Longrightarrow\ & \mathbf{w}\mathbf{x}_0 - b = 1 - r\|\mathbf{w}\| \\
\Longrightarrow\ & -1 = 1 - r\|\mathbf{w}\| \\
\Longrightarrow\ & r = \frac{2}{\|\mathbf{w}\|}
\end{aligned}
$$

Compute the distance from a point to the hyperplane

- Margin = 2/||w||
- The only variable we can change in this formula is the norm of w
  When ‖w‖=1 then margin=2
  When ‖w‖=2 then margin =1
  When ‖w‖=4 then margin =1/2
- The bigger the norm is, the smaller the margin.
- Maximizing the margin is the same thing as minimizing the norm of w
- The goal is to maximize the margin.
- Among all possible hyperplanes meeting the constraints, we will choose the hyperplane with the smallest ‖w‖
- because it is the one which will have the biggest margin.

Optimal hyperplane

$$\text{Minimize in } (\mathbf{w}, b)$$

$$\|\mathbf{w}\|$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x_i} + b) \geq 1$$

$$\text{(for any } i = 1, \ldots, n)$$

- This gives the above optimization problem
- Optimization problem is the problem of finding the best solution from all feasible solutions.
- Solving the equation, we get (w,b) for which ‖w‖ is the smallest possible and the constraints are met.
- This means we have the equation of the optimal hyperplane

Alternate way to calculate margin

Margin

Define the hyperplanes H such that:

$w \bullet x_i + b \geq +1$ when $y_i = +1$

$w \bullet x_i + b \leq -1$ when $y_i = -1$

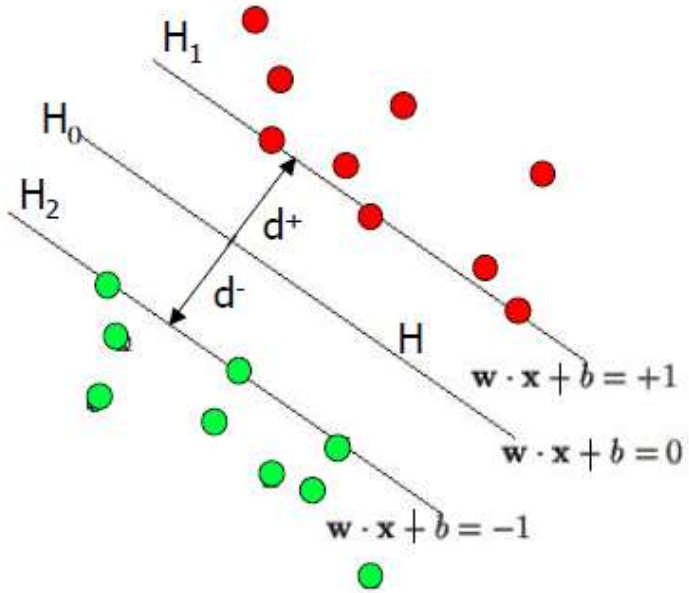
H_1 and H_2 are the planes:

H_1: $w \bullet x_i + b = +1$

H_2: $w \bullet x_i + b = -1$

The points on the planes H_1 and H_2 are the tips of the Support Vectors

The plane H_0 is the median in between, where $w \bullet x_i + b = 0$



d+ = the shortest distance to the closest positive point

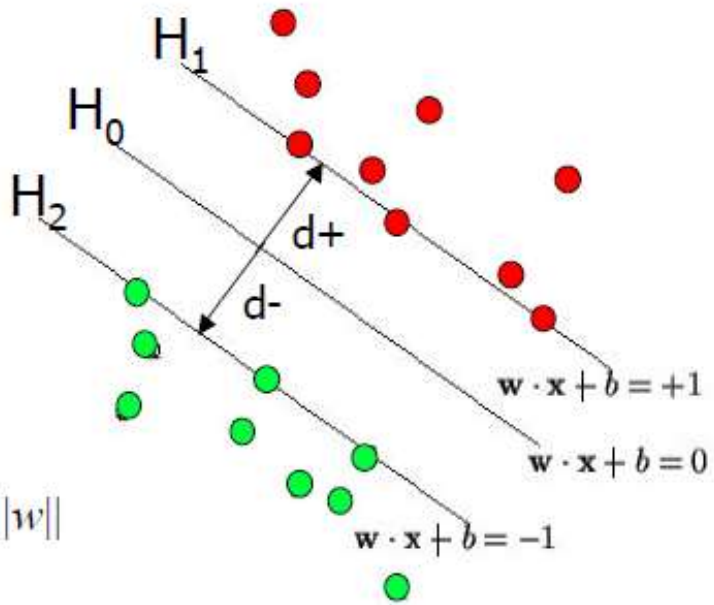
d- = the shortest distance to the closest negative point

The margin (gutter) of a separating hyperplane is d+ + d–.

- The distance from a point ⟦(x⟧_0,y_0) to a line: Ax+By+c = 0 is:
- |Ax_0 + By_0 +c|/sqrt(A2+B2)
- So, The distance between H0 and H1 is then:
- |w•x+b|/||w||=1/||w||
- So The total distance between H1 and H2 is thus: 2/||w||

Maximizing the margin

We want a classifier (linear separator) with as big a margin as possible.



Recall the distance from a point$(x_0.y_0)$ to a line: $Ax+By+c = 0$ is: $|Ax_0 +By_0 +c|/\text{sqrt}(A^2+B^2)$, so,
The distance between H_0 and H_1 is then:
$|w\bullet x+b|/||w||=1/||w||$, so

The total distance between H_1 and H_2 is thus: $2/||w||$

In order to maximize the margin, we thus need to minimize $||w||$. With the condition that there are no datapoints between H_1 and H_2:

$\mathbf{x}_i\bullet\mathbf{w}+b \geq +1$ when $y_i = +1$
$\mathbf{x}_i\bullet\mathbf{w}+b \leq -1$ when $y_i = -1$

Can be combined into: $y_i(\mathbf{x}_i\bullet\mathbf{w}) \geq 1$

Problem statement 3 : Find the best hyperplane

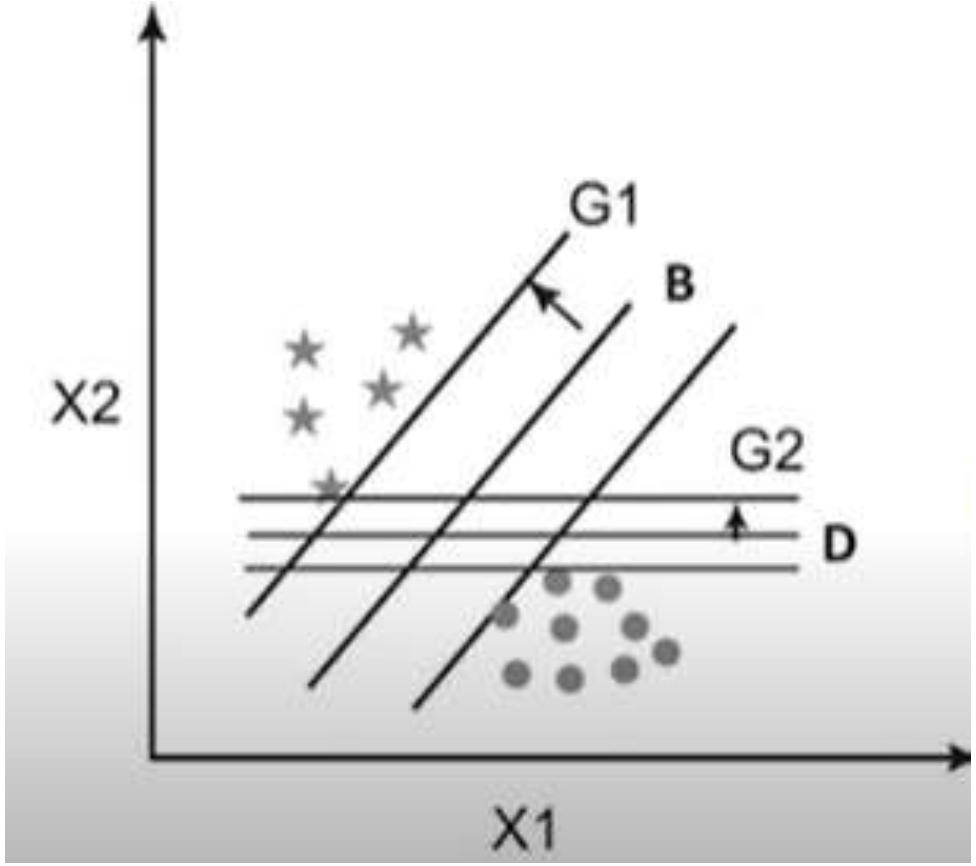
Find the best hyperplane between two hyperplanes

- Problem statement 3 : Given two hyperplanes for SVM classifier 1 and SVM Classifier 2, find the best hyperplane corresponding to the classifier.
- 5+2*x1+ 5*x2
- 5+20*x1+50*x2

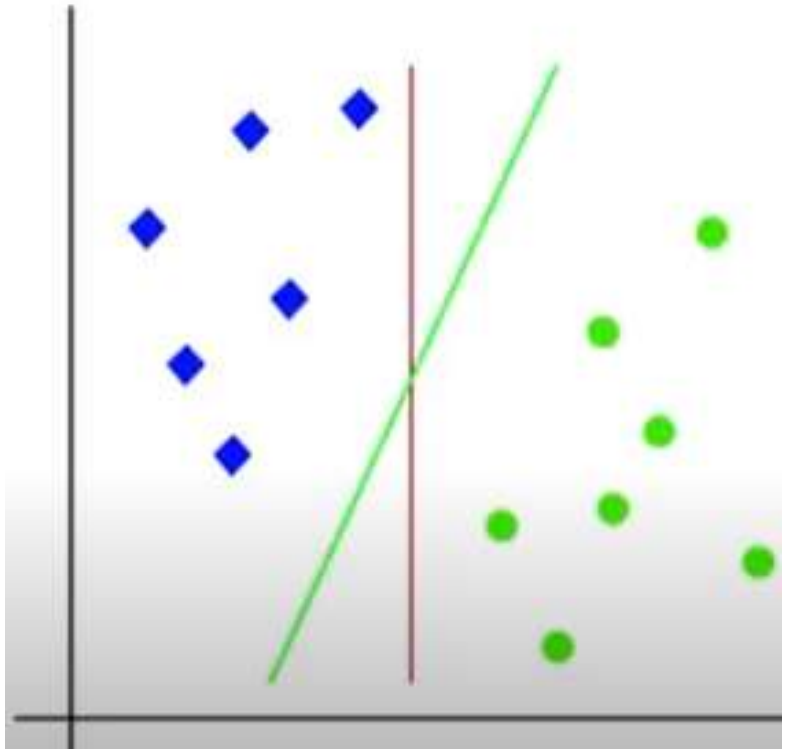
Which is the best hyperplane

- There are many possibilities to separate the two category classes
- But which is best among all?
- So the solution for the problem is HYPERPLANE
- Among these different hyperplanes, which is the best hyperplane

Which is the best hyperplane

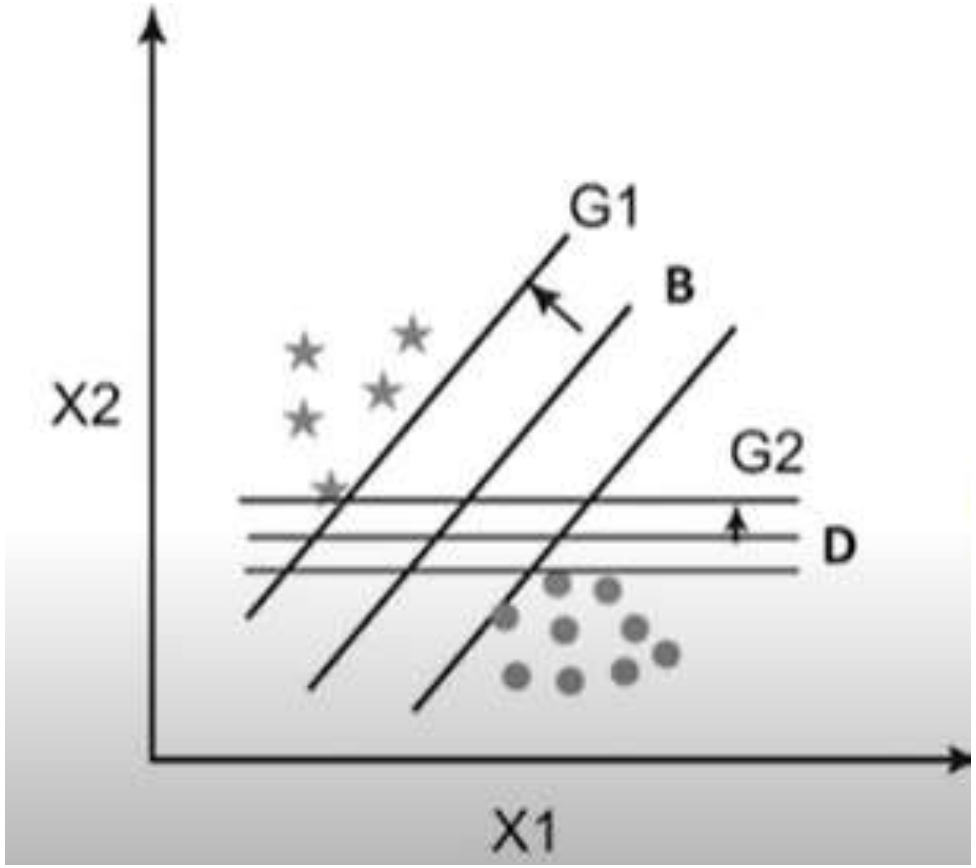


B or D?

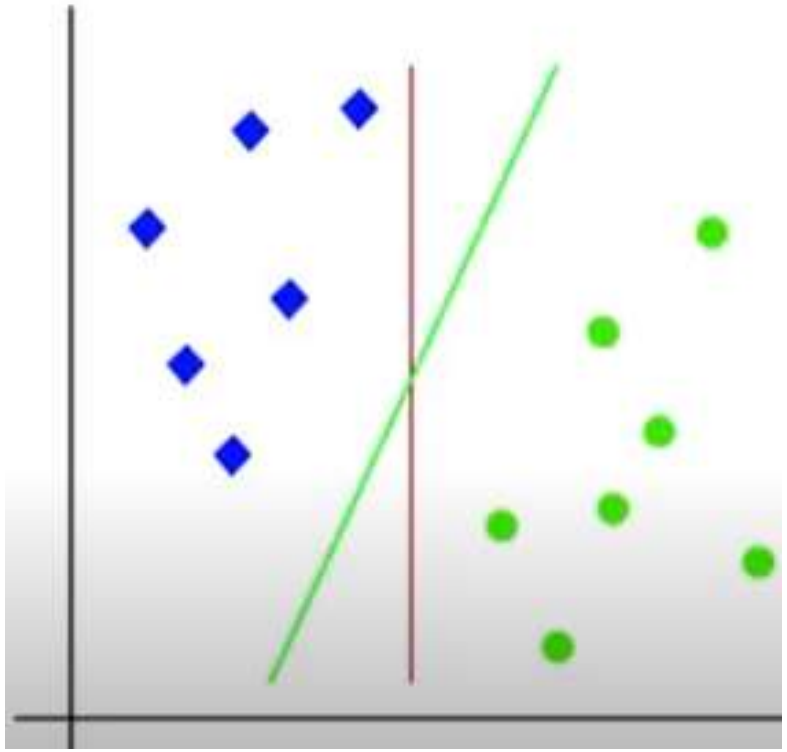


Red or green line?

Which is the best hyperplane

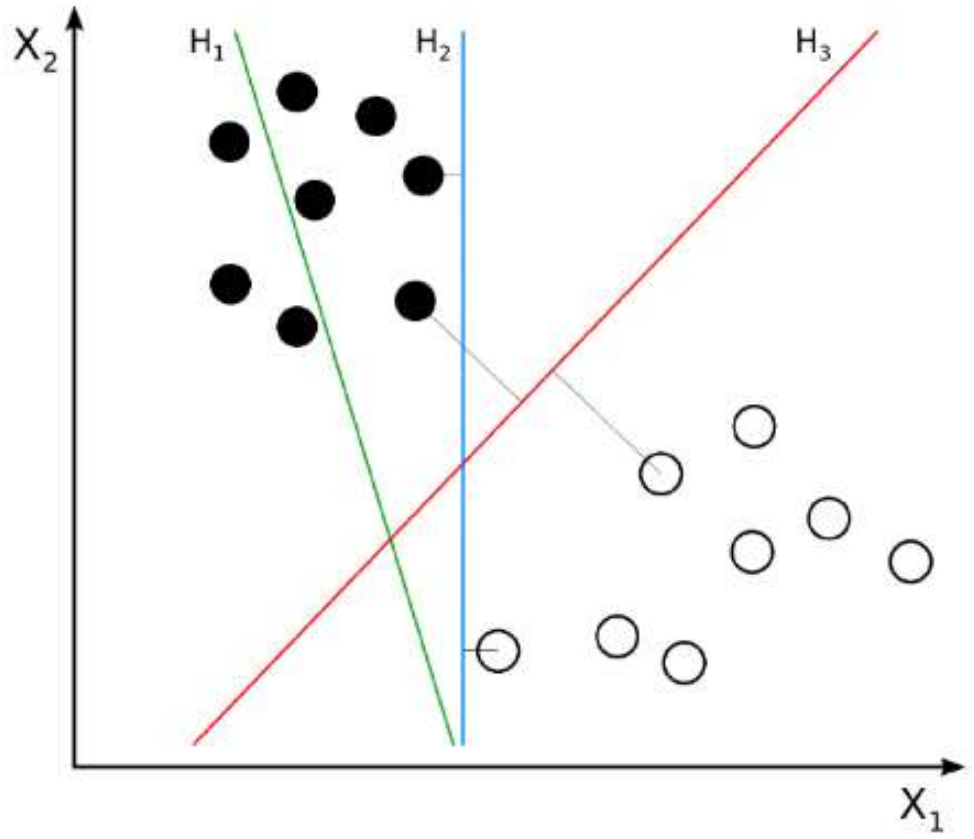


B or D? B



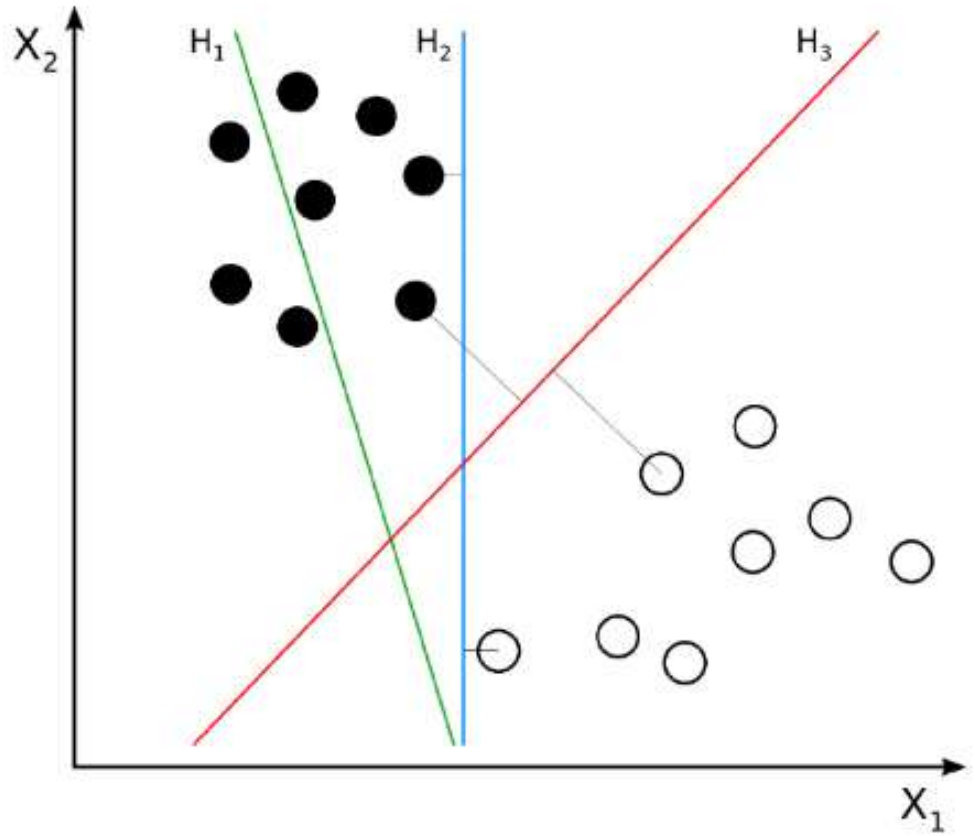
Red or green line? Green

Which is the best hyperplane



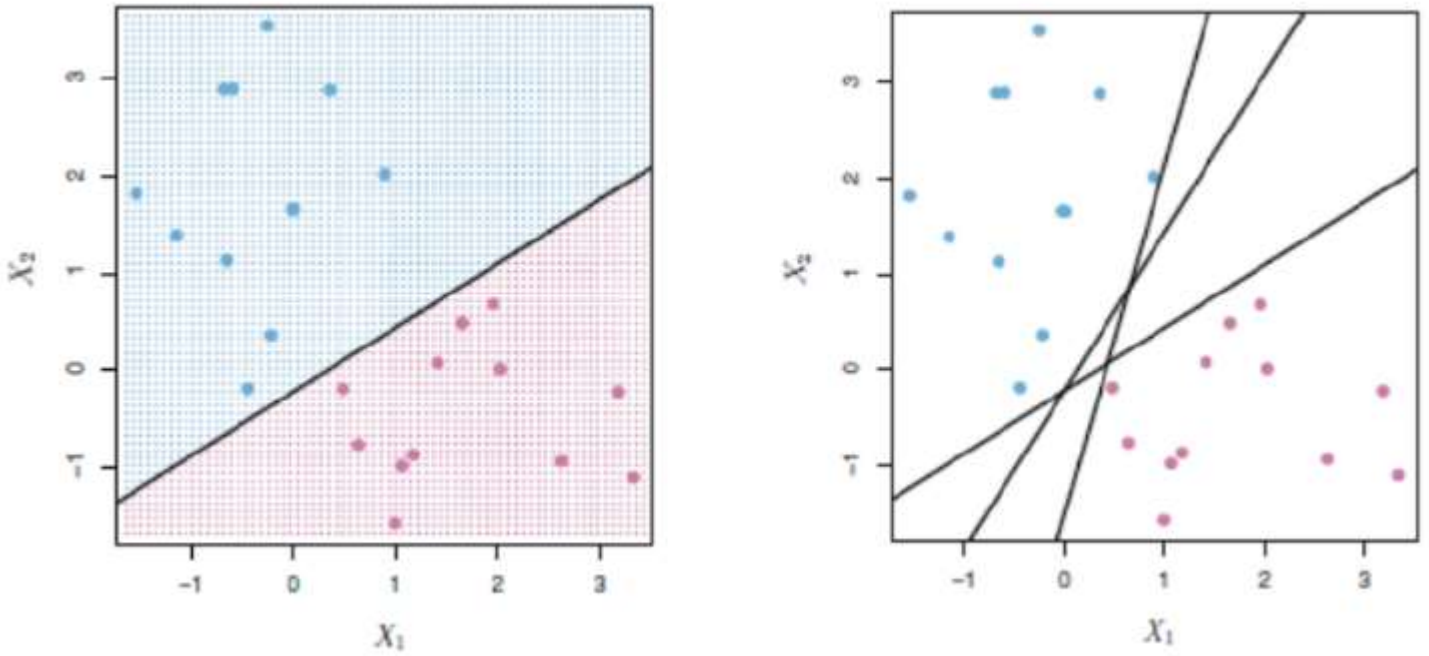
H1, H2 or H3?

Which is the best hyperplane

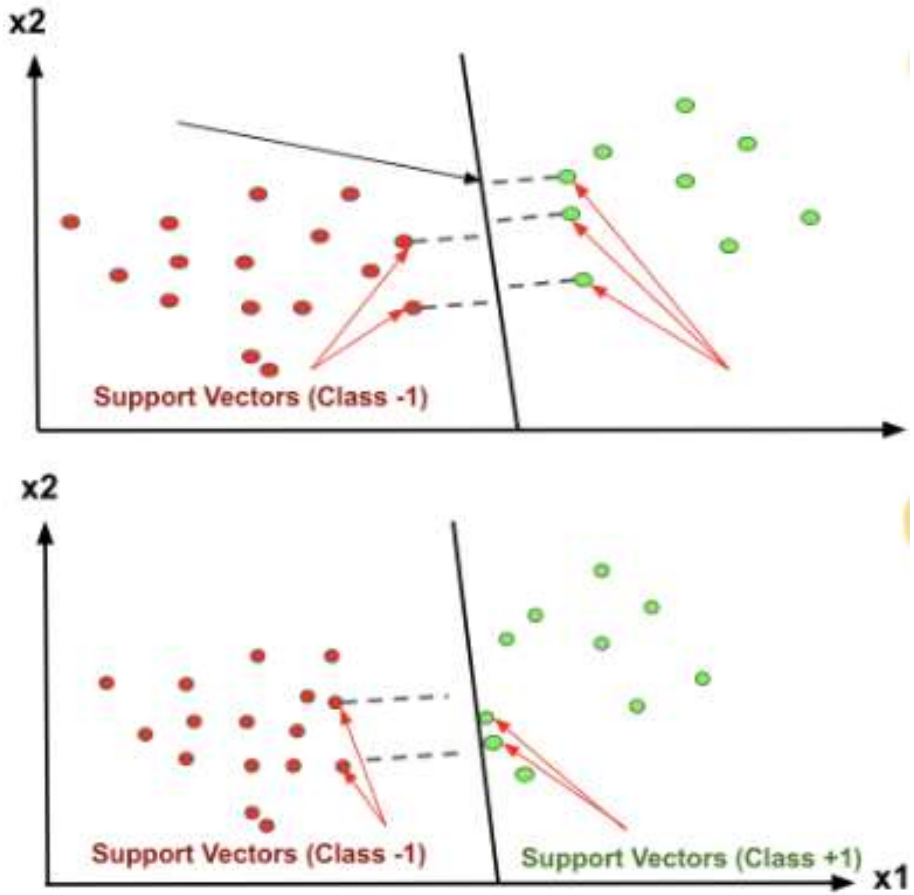


Red line (H3) has the maximum margin with the data points and also classified between the data properly
Blue line (H2) has the small margin with one data and large with another data
Green line (H1), has not classified between the data itself

Which is the best hyperplane



Which is the best hyperplane



Good Margin – All support vectors have the same distance with the maximal margin hyperplane

Bad margin – very close to either class -1 support vectors or class +1 support vectors

Support vectors

1. The data/vector points closest to the hyperplane are known as the support vector (SV) data points

- only support vectors are contributing to the result of the algorithm (SVM), other points are not.

2. If a data point is not a SV, removing it has no effect on the model.

3. Deleting the SV will then change the position of the hyperplane.

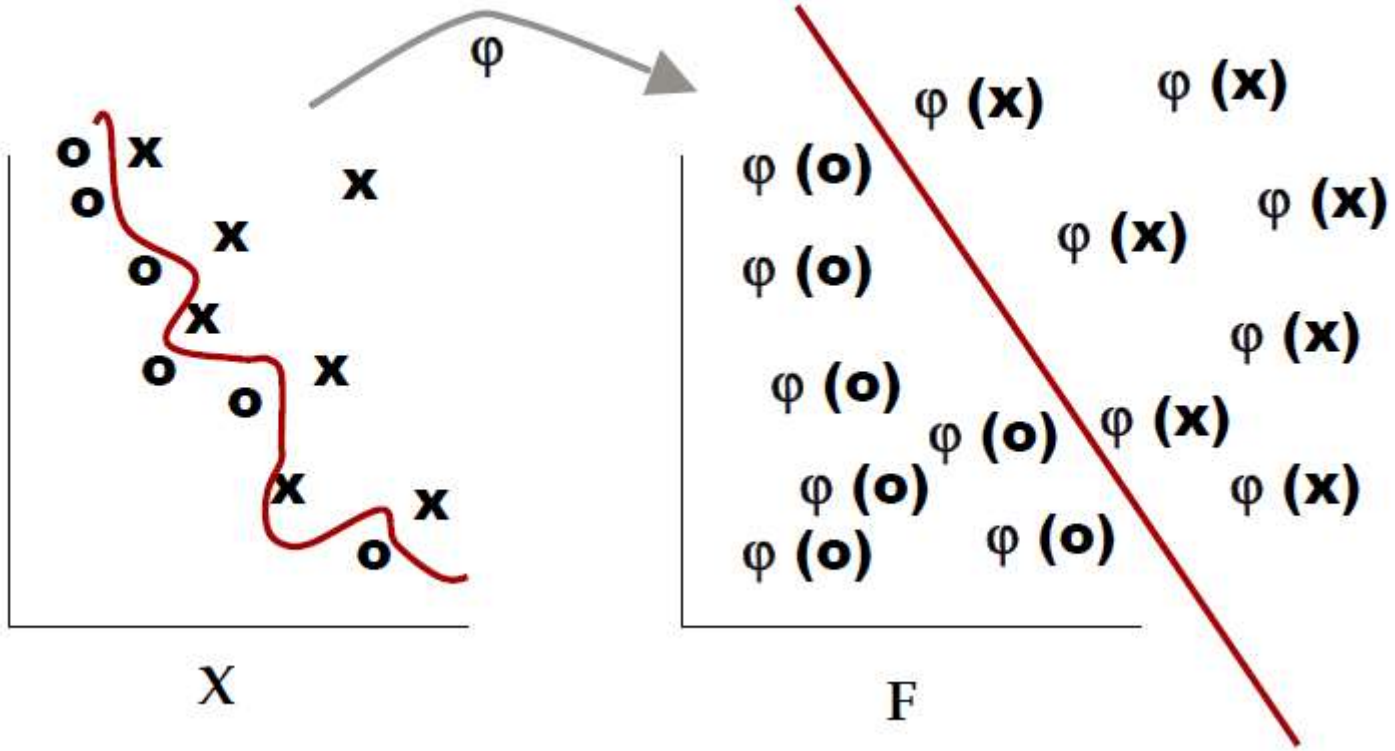
SVM -Summary of the history

- Next:
- Maximal Margin Classifier (1963 or 1979)
- Kernel Trick (1992)
- Soft Margin Classifier (1995)
- Support Vector Regression (1996)

Nonlinear Example – when Φ is non-trivial

Input space: the space where the point xi are located

Feature space: the space of f(xi) after transformation



Problem statement 4– Nonlinear SVM

Nonlinear Example – when Φ is non-trivial

- We are given the following positively labeled data points

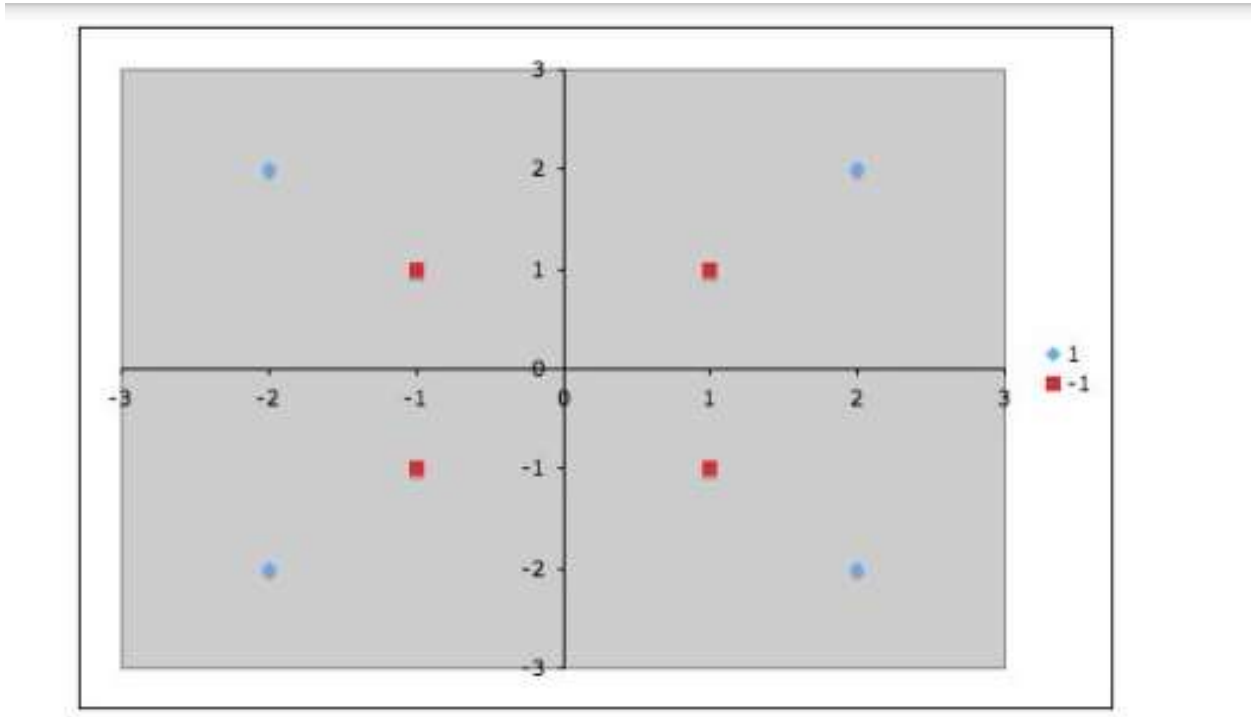
$$\left\{\begin{pmatrix}2\\2\end{pmatrix},\begin{pmatrix}2\\-2\end{pmatrix},\begin{pmatrix}-2\\-2\end{pmatrix},\begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

- And the following negatively labeled data points

$$\left\{\begin{pmatrix}1\\1\end{pmatrix},\begin{pmatrix}1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\end{pmatrix}\right\}$$

- Draw the hyperplane

Nonlinear Example - Graph



Nonlinearly separable sample data points
Blue diamonds are positive examples
Red squares are negative examples

Nonlinear Example

- Find a separating hyperplane that accurately discriminates the two classes.
- It is obvious that no such hyperplane exists in the input space (that is, in the space in which the original input data exist)
- Therefore, we must use a nonlinear SVM
- That is, one whose mapping function Φ is a nonlinear mapping from input space into some feature space).
- Define Φ

Find the hyperplane

Steps

1. Plot the graph and identify whether the data is linearly separable. If not, use non linear SVM
2. Define a mapping function Φ which is a nonlinear mapping from input space into some feature space
3. Identify the support vectors or nearest data points by inspection on the graph
4. Construct augmented vectors by adding the bias input and solve the system of linear equations for αi
5. Compute the weight vector, wi= Σ αi * si
6. Draw the hyperplane

Nonlinear Example

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \quad (1)$$

Rewrite the data in feature space

Positive Examples

Negative examples

Nonlinear Example

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \quad (1)$$

Rewrite the data in feature space

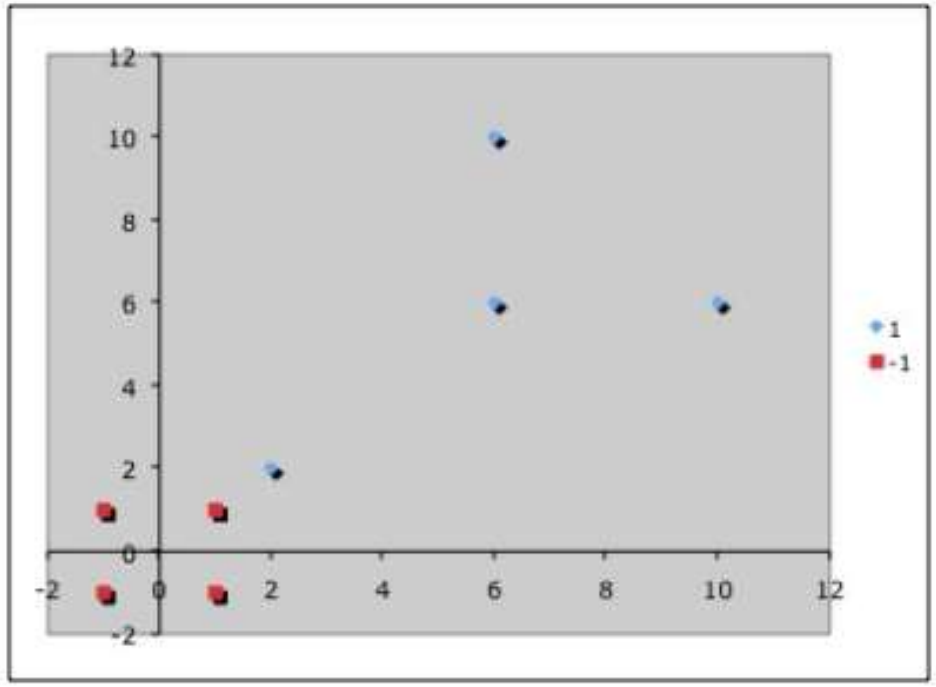
Positive Examples

(2 2) (10 6) (6 6) (6 10)

Negative examples

$$\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}$$

Nonlinear Example- The data represented in feature space



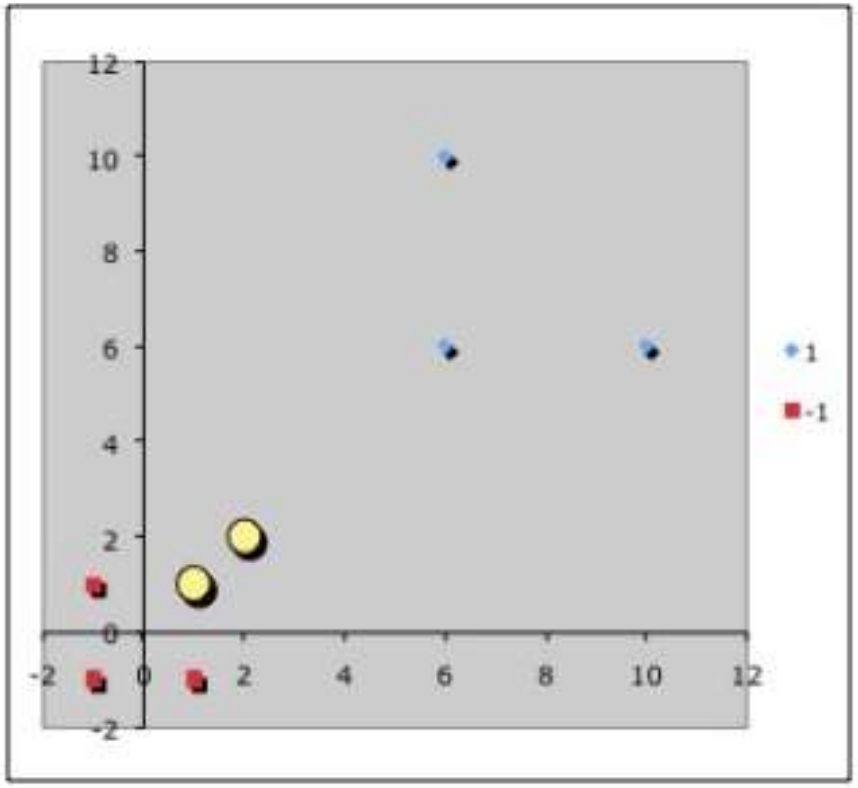
Identify the support vectors

Nonlinear Example

- Identify the support vectors

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, s_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$$

Nonlinear Example – Support vectors



The two support vectors (in feature space) are marked as yellow circles.

Nonlinear Example

- Identify the support vectors

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, s_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$$

- Use vectors augmented with a 1 as a bias input
- Now given the [augmented] support vectors, find values for the αi

$$\begin{aligned} \alpha_1 \Phi_1(s_1) \cdot \Phi_1(s_1) + \alpha_2 \Phi_1(s_2) \cdot \Phi_1(s_1) &= -1 \\ \alpha_1 \Phi_1(s_1) \cdot \Phi_1(s_2) + \alpha_2 \Phi_1(s_2) \cdot \Phi_1(s_2) &= +1 \end{aligned}$$

- As per equation 1 , this reduces to

$$\begin{aligned} \alpha_1 \tilde{s_1} \cdot \tilde{s_1} + \alpha_2 \tilde{s_2} \cdot \tilde{s_1} &= -1 \\ \alpha_1 \tilde{s_1} \cdot \tilde{s_2} + \alpha_2 \tilde{s_2} \cdot \tilde{s_2} &= +1 \end{aligned}$$

Nonlinear Example

- Even though $\Phi 1$ is a nontrivial function, both s1 and s2 map to themselves under $\Phi 1$.
- This will not be the case for other inputs
- Computing the dot products results in
- $3\alpha 1 + 5\alpha 2 = -1$
- $5\alpha 1 + 9\alpha 2 = +1$
- Find solution to this system of equations

Nonlinear Example

- Even though $\Phi 1$ is a nontrivial function, both s1 and s2 map to themselves under $\Phi 1$.
- This will not be the case for other inputs
- Computing the dot products results in
- $3\alpha 1 + 5\alpha 2 = -1$
- $5\alpha 1 + 9\alpha 2 = +1$
- Find solution to this system of equations
- $\alpha 1 = -7$ and $\alpha 2 = 4$.
- Looking at the discriminating hyperplane in input space that corresponds to these α.
- $\tilde{w} = \Sigma_i \alpha i * \tilde{s} i$

Nonlinear Example

$$\tilde{w} = \sum_i \alpha_i \tilde{s}_i$$

$$= -7 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 4 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}$$

$$= \begin{pmatrix} 1 \\ 1 \\ -3 \end{pmatrix}$$

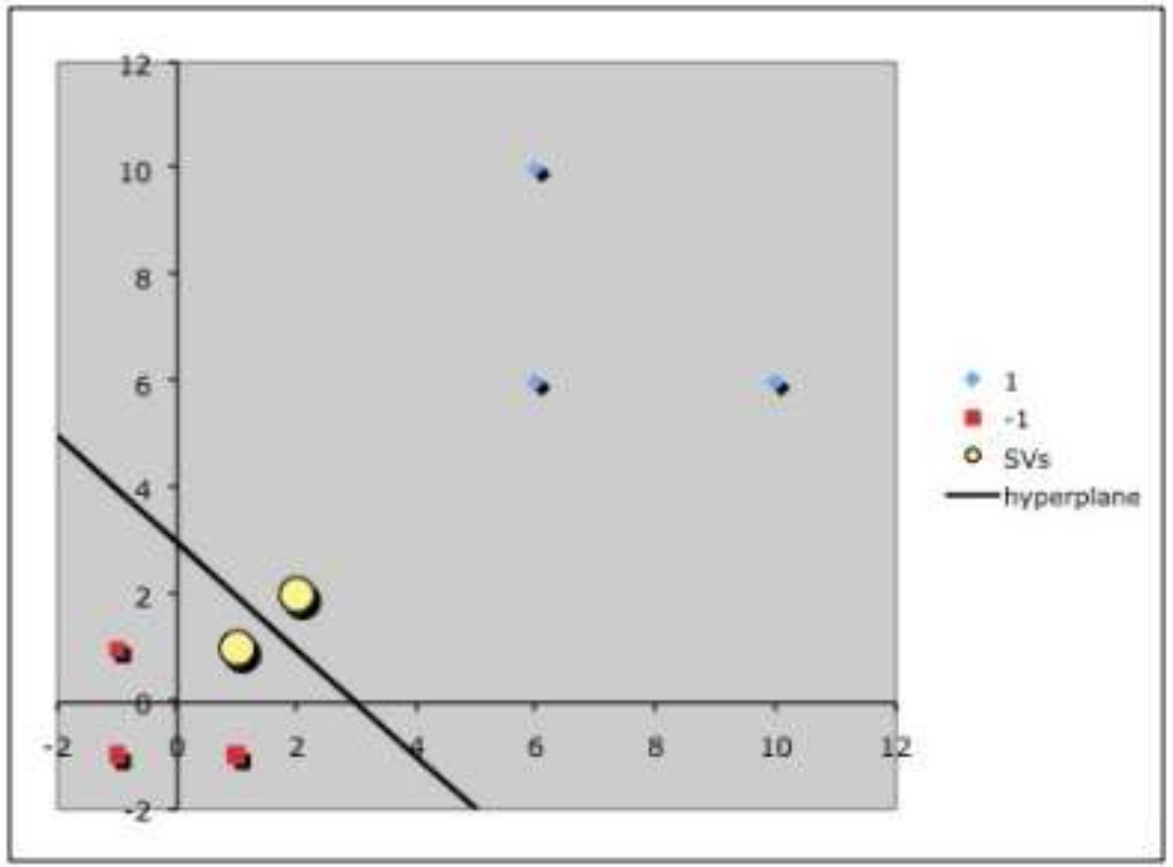
The separating hyperplane equation y = wx + b with w = (1 1) and b = −3.

b= -3 and $w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Plot the line to get the expected decision surface

Nonlinear Example

The discriminating hyperplane corresponding to the values $\alpha 1 = -7$ and $\alpha 2 = 4$



Problem statement 5 – SVM classification

Nonlinear Example - Using the SVM

- How to use the SVM model to classify data?
- Given x, the classification f(x) is given by the equation

$$f(x) = \sigma\left(\sum_i \alpha_i \Phi(s_i) \cdot \Phi(x)\right) \qquad (2)$$

- where σ(z) returns the sign of z
- Classify the point x = (4, 5) using the mapping function of Eq. 1

$$\Phi_1\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \qquad (1)$$

Nonlinear Example – Same Problem Statement

- We are given the following positively labeled data points

$$\left\{\begin{pmatrix}2\\2\end{pmatrix},\begin{pmatrix}2\\-2\end{pmatrix},\begin{pmatrix}-2\\-2\end{pmatrix},\begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

- And the following negatively labeled data points

$$\left\{\begin{pmatrix}1\\1\end{pmatrix},\begin{pmatrix}1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\end{pmatrix}\right\}$$

- Classify the point x = (4, 5)
- We have already drawn the hyperplane

Rewrite(4,5) in feature space

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \quad (1)$$

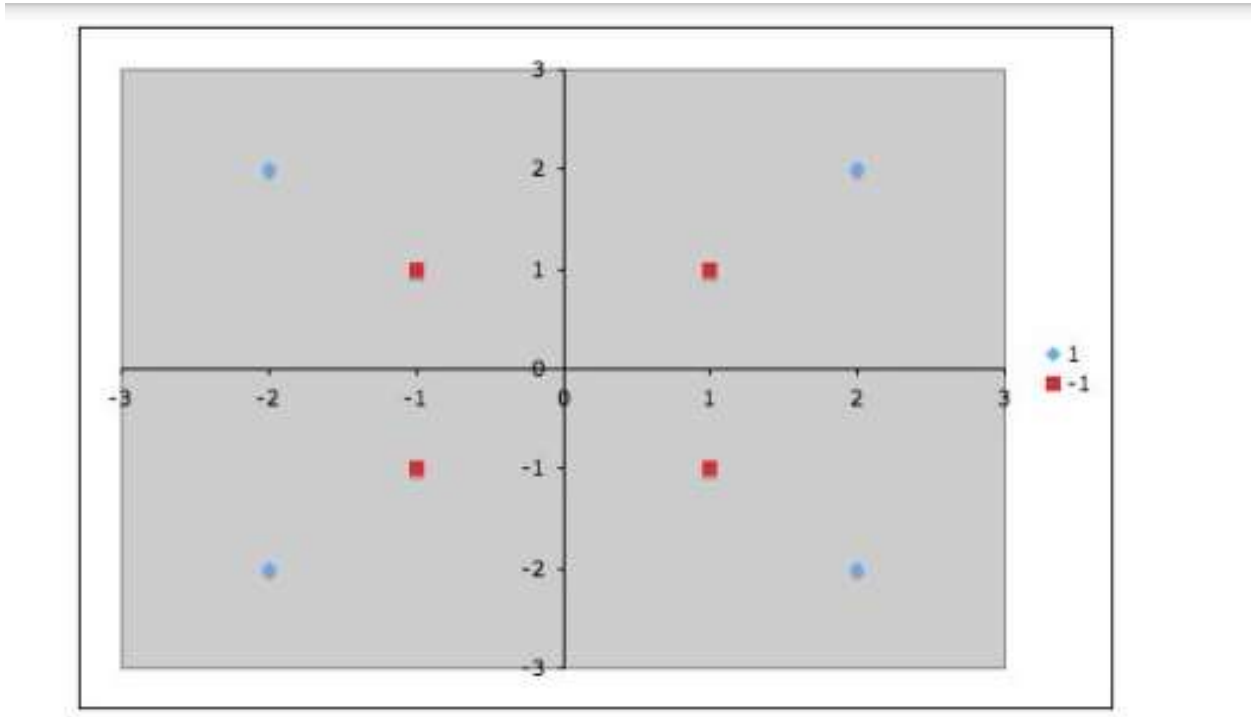
- Rewrite (4,5) in feature space
- Φ(4,5) =

Rewrite(4,5) in feature space

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \quad (1)$$

- Rewrite (4,5) in feature space
- Φ(4,5) = (0, 1)

Nonlinear Example - Graph



Nonlinearly separable sample data points
Blue diamonds are positive examples
Red squares are negative examples

Nonlinear Example

$$\Phi_1 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 4 - x_2 + |x_1 - x_2| \\ 4 - x_1 + |x_1 - x_2| \end{pmatrix} & \text{if } \sqrt{x_1^2 + x_2^2} > 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} & \text{otherwise} \end{cases} \quad (1)$$

Rewrite (4,5) in feature space
Φ(4,5) = (0,1)

Rewrite the data in feature space

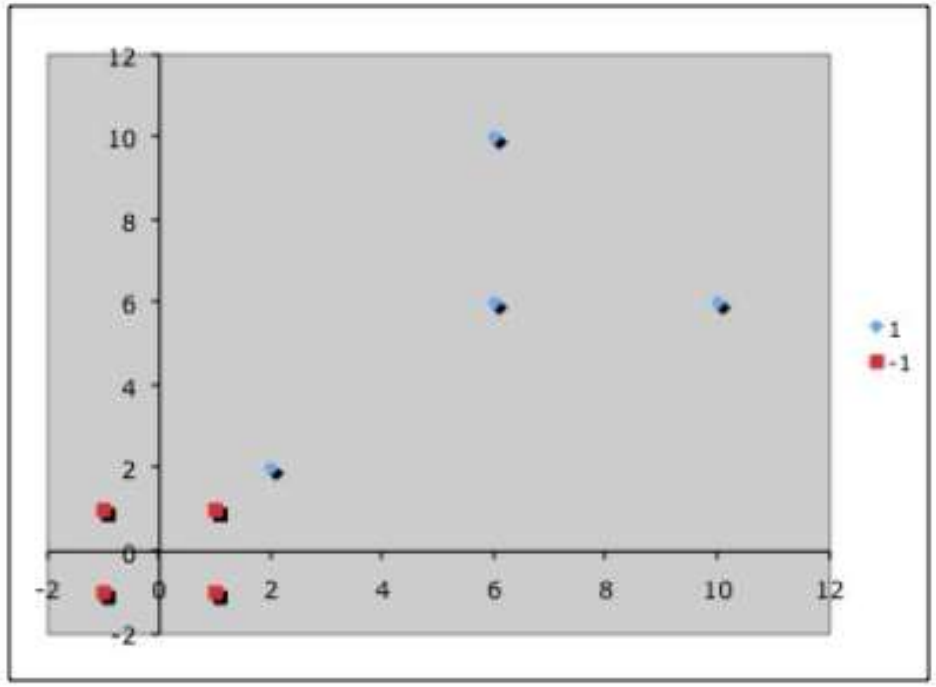
Positive Examples

(2 2) (10 6) (6 6) (6 10)

Negative examples

$$\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 1 \end{pmatrix} \right\}$$

Nonlinear Example- The data represented in feature space



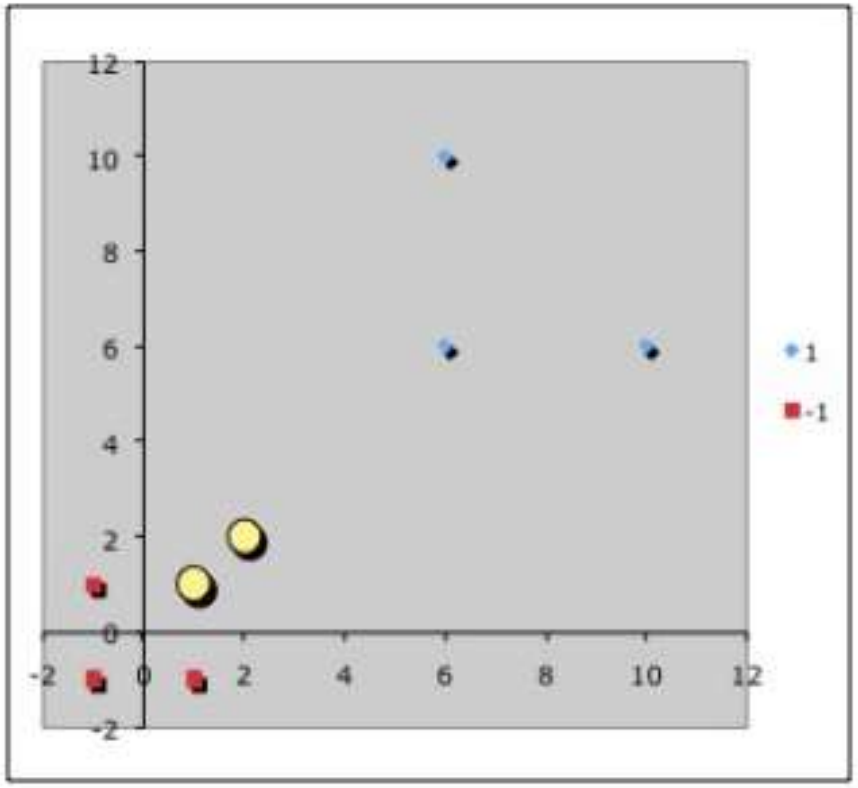
Identify the support vectors

Nonlinear Example

- Identify the support vectors

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, s_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$$

Nonlinear Example – Support vectors



The two support vectors (in feature space) are marked as yellow circles.

Nonlinear Example

$$\tilde{w} = \sum_i \alpha_i \tilde{s}_i$$

$$= -7 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 4 \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}$$

$$= \begin{pmatrix} 1 \\ 1 \\ -3 \end{pmatrix}$$

α1 = −7 and α2 = 4

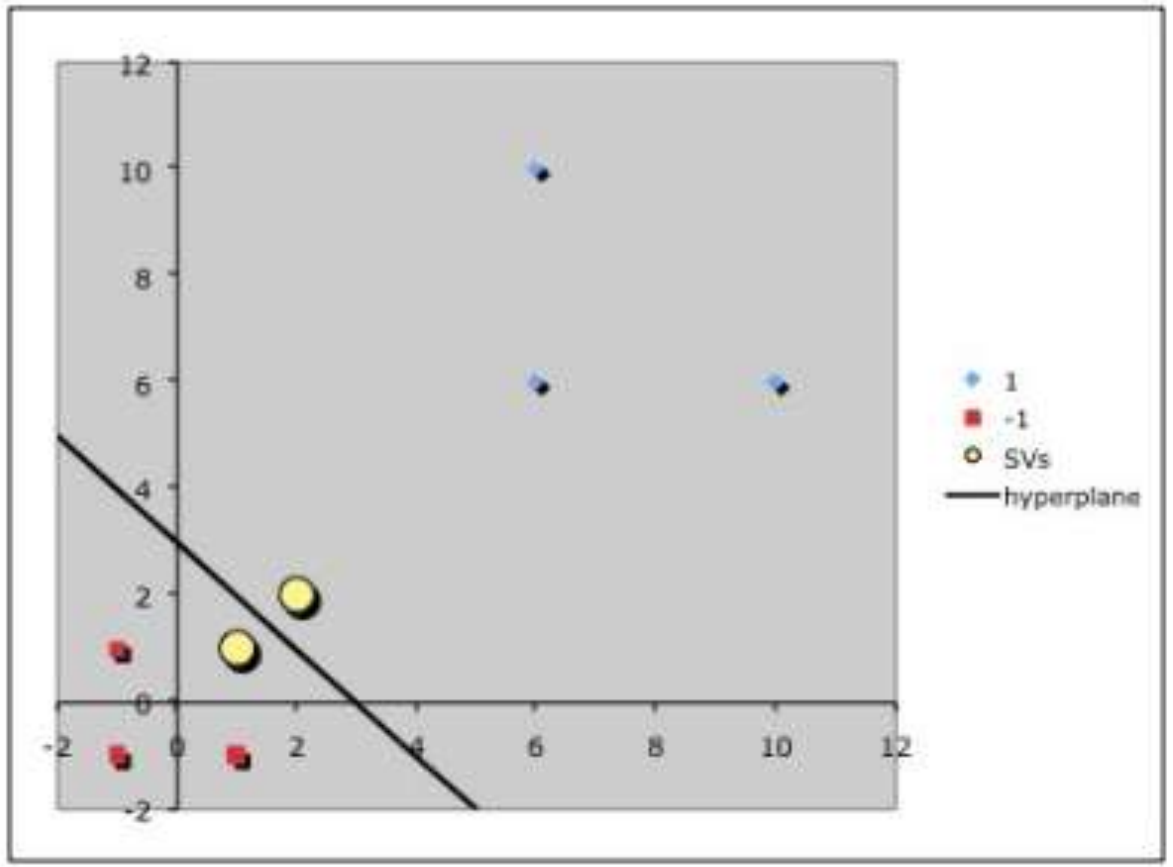
The separating hyperplane equation y = wx + b with w = (1 1) and b = −3.

b= -3 and $w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Plot the line to get the expected decision surface

Nonlinear Example

The discriminating hyperplane corresponding to the values $\alpha 1 = -7$ and $\alpha 2 = 4$



Nonlinear Example - Using the SVM

Support vectors and alpha values as identified earlier are

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, s_2 = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \right\}$$

α1 = −7 and α2 = 4

Classify the point x = (4, 5)

Given x, the classification f(x) is given by the equation

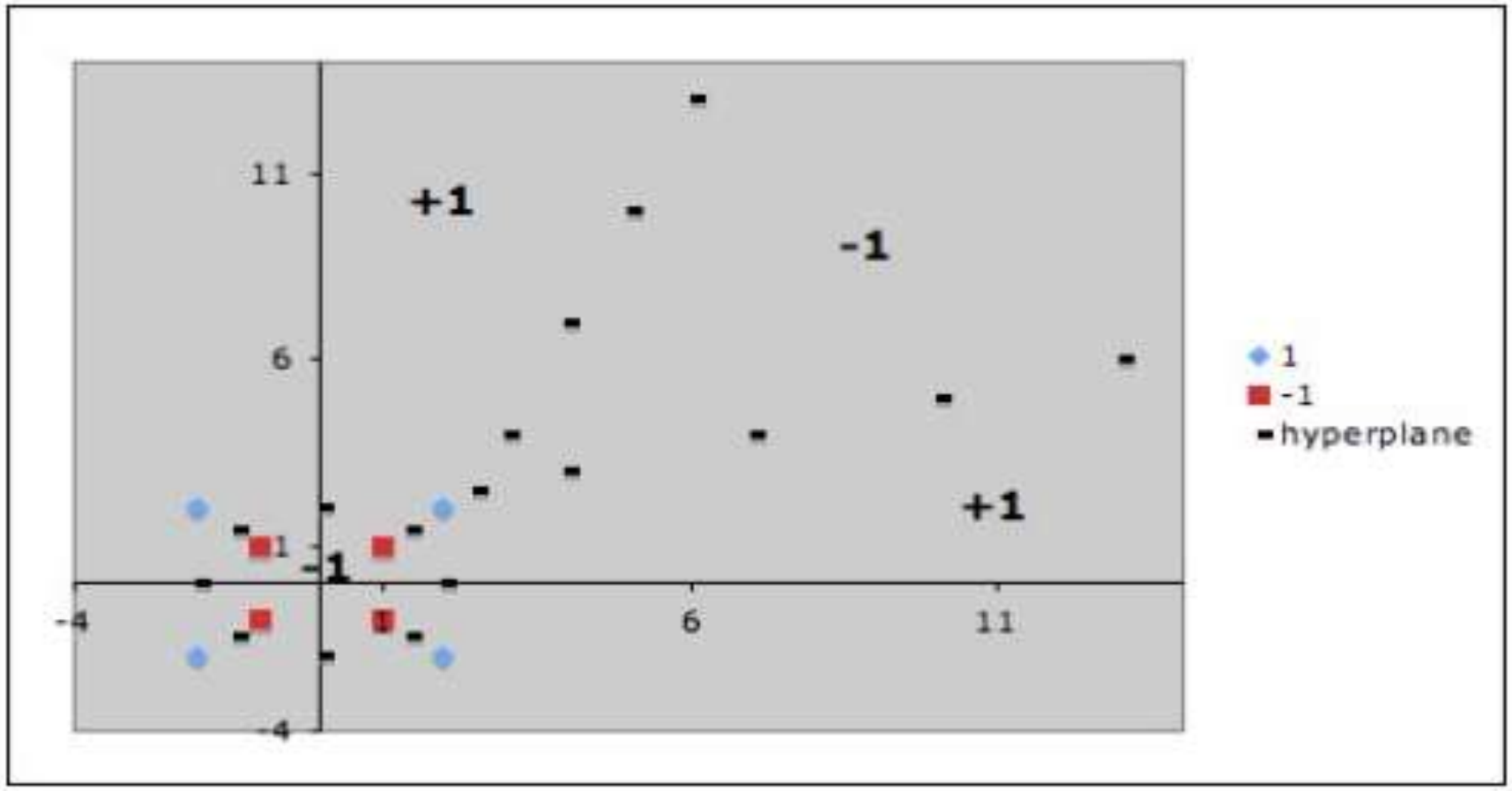
$$f(x) = \sigma\left(\sum_i \alpha_i \Phi(s_i) \cdot \Phi(x)\right) \tag{2}$$

$$\begin{aligned} f\begin{pmatrix} 4 \\ 5 \end{pmatrix} &= \sigma\left(-7\Phi_1\begin{pmatrix} 1 \\ 1 \end{pmatrix} \cdot \Phi_1\begin{pmatrix} 4 \\ 5 \end{pmatrix} + 4\Phi_1\begin{pmatrix} 2 \\ 2 \end{pmatrix} \cdot \Phi_1\begin{pmatrix} 4 \\ 5 \end{pmatrix}\right) \\ &= \sigma\left(-7\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} + 4\begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}\right) \\ &= \sigma(-2) \end{aligned}$$

Nonlinear Example - Using the SVM

- x = (4, 5) is classified as negative
- Looking at the input space, we might be tempted to think this is not a reasonable classification
- However, this is as per our model and our model is consistent with all the training data.
- As always, there are no guarantees on generalization accuracy
- If we are not happy about our generalization, the likely culprit is our choice of Φ.
- if we map our discriminating hyperplane (which lives in feature space) back into input space, we can see the effective decision surface of our model
- We may or may not be able to improve generalization accuracy by choosing a different Φ
- however, there is another reason to revisit our choice of mapping function

Nonlinear Example - Using the SVM



The decision surface in input space corresponding to Φ1.

Problem Statement 6: Alternate mapping function

Alternate mapping function for non-linear SVM

- Definition of Φ in Eq. 1 preserved the number of dimensions.
- That is, input and feature spaces are the same size.
- However, in order to effectively separate the data, we must use a feature space that is of (sometimes very much) higher dimension than our input space.
- Consider an alternative mapping function

$$\Phi_2 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ \frac{(x_1^2+x_2^2)-5}{3} \end{pmatrix} \qquad (3)$$

- This mapping function transforms data from 2-dimensional input space to 3-dimensional feature space.
- Using this alternative mapping, the data in the new feature space after applying to the same data points

Alternate mapping function for non-linear SVM

Positive examples in the transformed 3D space

$$\left\{\begin{pmatrix}2\\2\\1\end{pmatrix},\begin{pmatrix}2\\-2\\1\end{pmatrix},\begin{pmatrix}-2\\-2\\1\end{pmatrix},\begin{pmatrix}-2\\2\\1\end{pmatrix}\right\}$$

Positive examples in the original 2D space

$$\left\{\begin{pmatrix}2\\2\end{pmatrix},\begin{pmatrix}2\\-2\end{pmatrix},\begin{pmatrix}-2\\-2\end{pmatrix},\begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

Negative examples in the transformed 3D space

$$\left\{\begin{pmatrix}1\\1\\-1\end{pmatrix},\begin{pmatrix}1\\-1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\\-1\end{pmatrix}\right\}$$

Negative examples in the original 2D space

$$\left\{\begin{pmatrix}1\\1\end{pmatrix},\begin{pmatrix}1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\end{pmatrix}\right\}$$

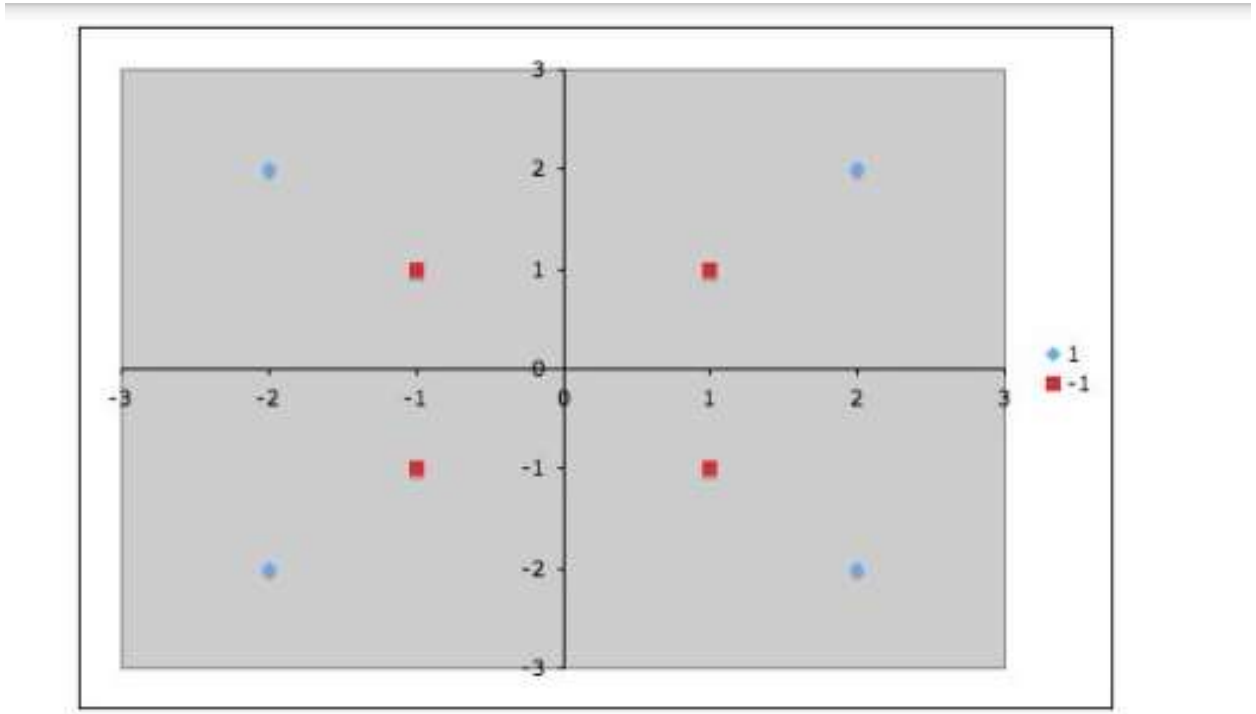
Positive examples in the transformed 3D space (x1, x2, x3)

x1= [2, 2, -2, -2,]
x2=[2, -2, -2, 2,]
x3=[1, 1, 1, 1]

Negative examples in the original 2D space (x1, x2, x3)

x1n=[1, 1, -1, -1]
x2n=[1, -1, -1, 1]
x3n = [-1, -1, -1, -1]

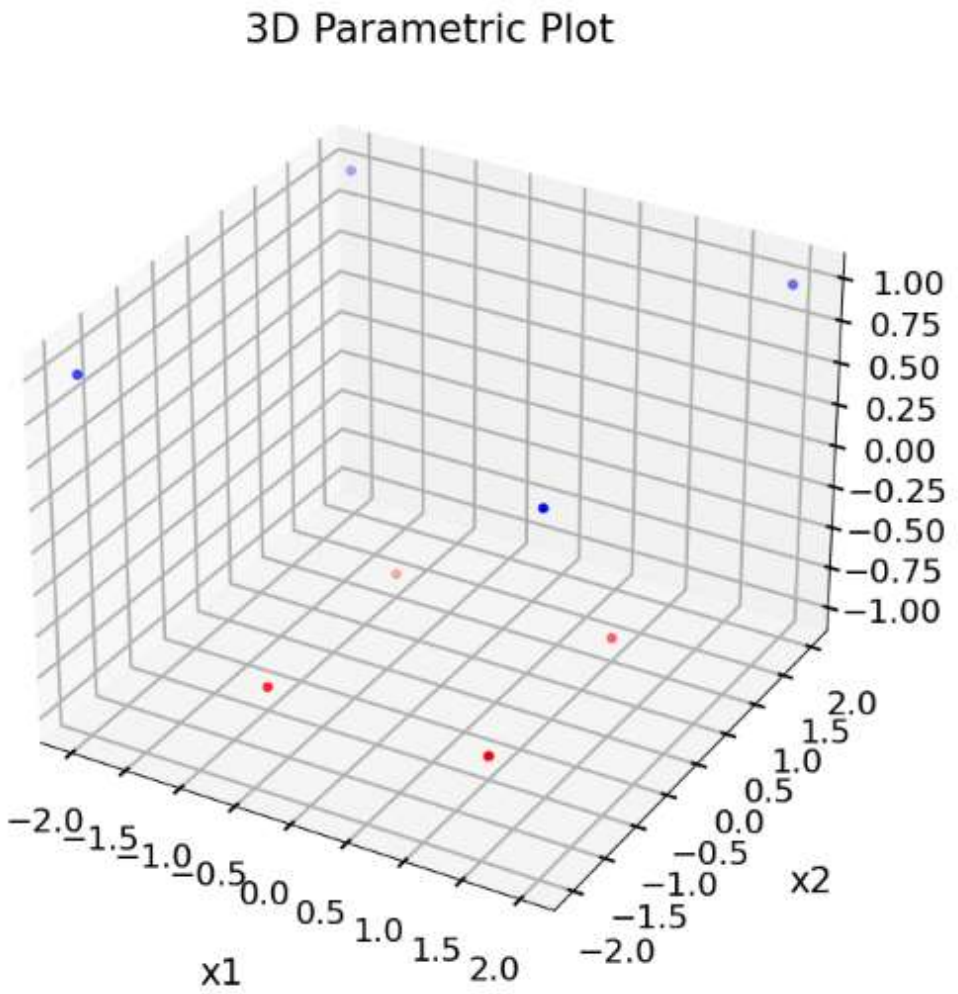
Nonlinear Example - Graph Original 2D Space



Nonlinearly separable sample data points
Blue diamonds are positive examples
Red squares are negative examples

Alternate mapping function for non-linear SVM-3D Space

```
import numpy as np
from mpl_toolkits import mplot3dimport
matplotlib.pyplot as plt
fig = plt.figure(figsize = (8,8))
ax =plt.axes(projection='3d')
#ax.grid()
x1= [2, 2, -2, -2, ]
x2=[2, -2, -2, 2, ]
x3=[1, 1, 1, 1]
x1n=[ 1, 1, -1, -1]
x2n=[ 1, -1, -1, 1]
x3n = [-1, -1, -1, -1]
#ax.plot3D(x1, x2, x3, 'red')
ax.scatter(x1, x2, x3,  c='Blue')
ax.scatter(x1n, x2n, x3n, c='Red')
ax.set_title('3D Parametric Plot')
# Set axes label
ax.set_xlabel('x1', labelpad=30)
ax.set_ylabel('x2', labelpad=30)
ax.set_zlabel('x3', labelpad=30)
plt.show()
```



Alternate mapping function for non-linear SVM

- The decision boundary in 3D space is a plane here not a line
- So, identify support vectors. How many support vectors?
- Solve for αi
- Compute weight vector

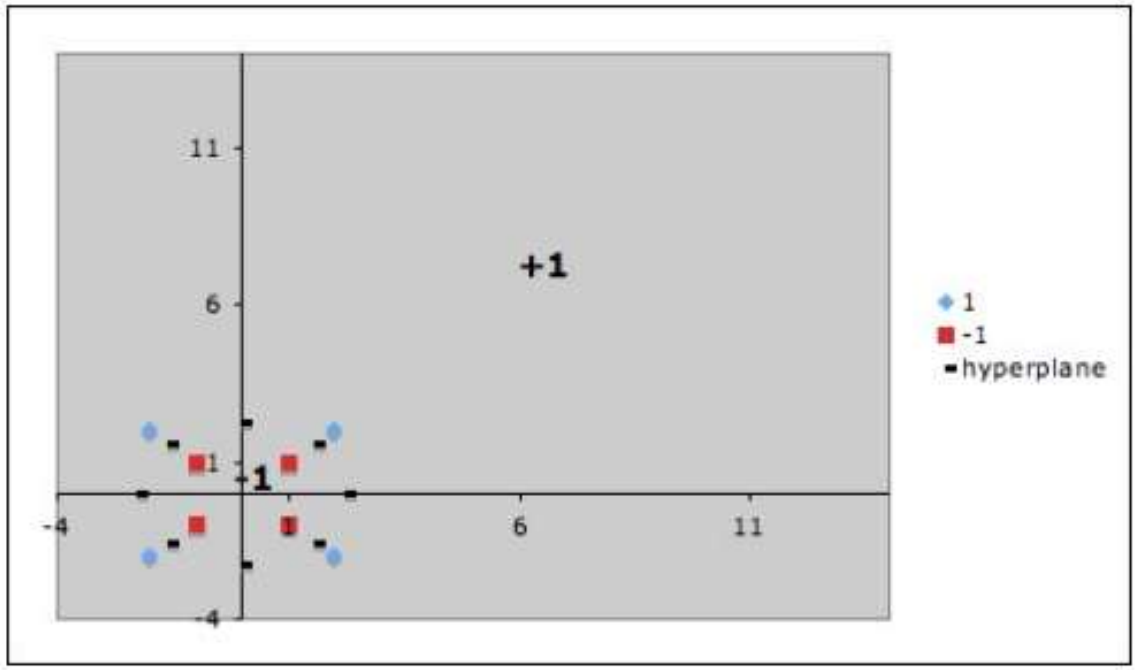
Alternate mapping function for non-linear SVM

- All 8 of the examples will be support vectors
- With αi = 1/46 for the positive support vectors and αi = −7/46 for the negative ones.
- A consequence of this mapping is that we do not need to use augmented vectors
- because the hyperplane in feature space goes through the origin
- y = wx+b , b = 0 $w = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$
- Therefore, the discriminating feature, is x3 (for classification)
- Eq. 2 reduces to f(x) = σ(x3) $$f(x) = \sigma\left(\sum_i \alpha_i \Phi(s_i) \cdot \Phi(x)\right) \quad (2)$$
- Plot the decision surface induced in the input space for this new mapping function

Alternate mapping function for non-linear SVM

The decision surface in input space corresponding Φ2

$$\Phi_2 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ \frac{(x_1^2+x_2^2)-5}{3} \end{pmatrix} \qquad (3)$$



Positive examples in the transformed 3D space

$$\left\{\begin{pmatrix}2\\2\\1\end{pmatrix},\begin{pmatrix}2\\-2\\1\end{pmatrix},\begin{pmatrix}-2\\-2\\1\end{pmatrix},\begin{pmatrix}-2\\2\\1\end{pmatrix}\right\}$$

Positive examples in the original 2D space

$$\left\{\begin{pmatrix}2\\2\end{pmatrix},\begin{pmatrix}2\\-2\end{pmatrix},\begin{pmatrix}-2\\-2\end{pmatrix},\begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

Negative examples in the transformed 3D space

$$\left\{\begin{pmatrix}1\\1\\-1\end{pmatrix},\begin{pmatrix}1\\-1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\\-1\end{pmatrix}\right\}$$

Negative examples in the original 2D space

$$\left\{\begin{pmatrix}1\\1\end{pmatrix},\begin{pmatrix}1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\end{pmatrix}\right\}$$

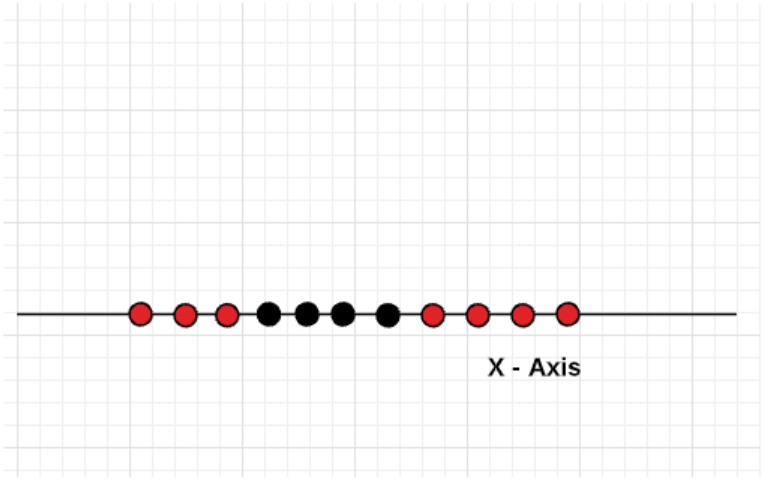
Kernel functions

The Kernel Concept

- Non-linear data set are difficult to be separated using a linear hyperplane.
- SVM algorithm is related to finding the hyperplane which separates the data based on maximum margin.
- “Is there a way in which a non-linear data set or data set which is linearly inseparable can be represented in another form such that the data becomes linearly separable?”
- The idea is to apply a function that transforms the data in such a manner that the data becomes linearly separable.
- In case of SVM algorithm, data becomes linearly separable by applying maximum margin

Kernel Functions - Non linear Transformations

- Example of data in 1 dimension which is not easy to separate
- Adding another dimension makes it easy
- Add another dimension to the data by using the function Y = X^2 (X-squared).
- After applying the kernel function (Y = X^2), data becomes linearly separable



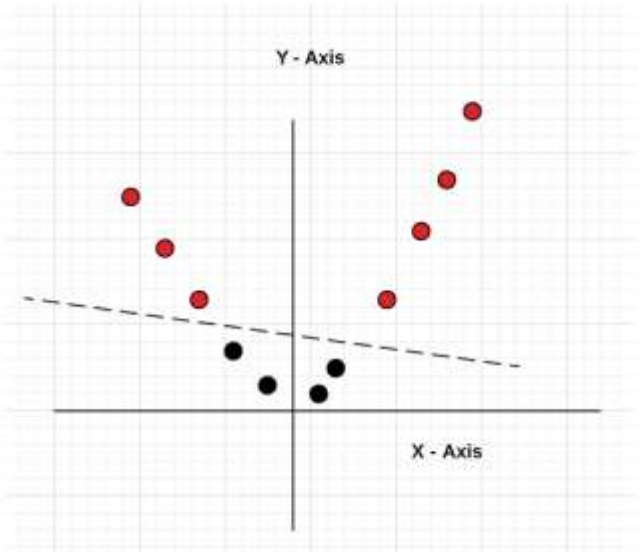
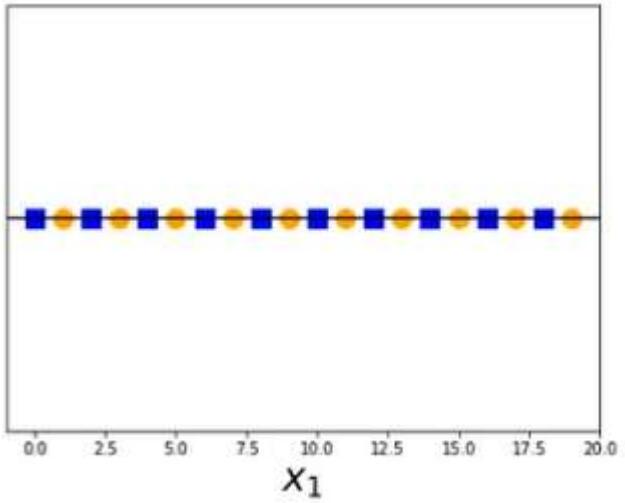
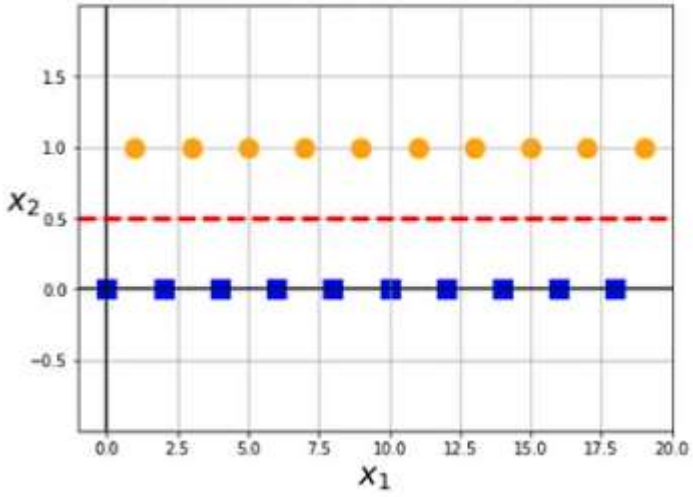


Fig1: Linearly inseparable data in 1-dimension Fig2: Applying kernel method to represent data using 2-dimensions

Kernel functions - Non linear Transformations

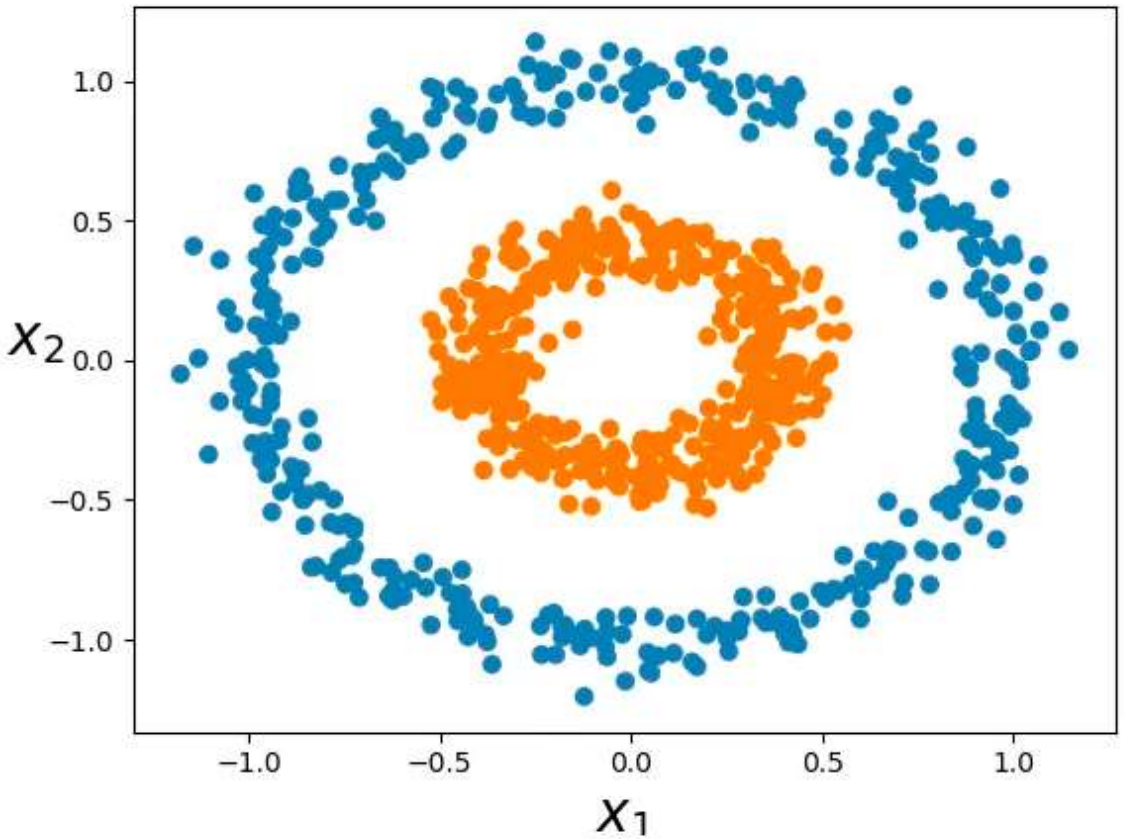
- Apply the transformation $\phi(x) = x \bmod 2$
- This transformation allows to linearly separate the even and odd X1 values in 2 dimensions





Note:There are many possible functions that can map the data to any number of higher dimensions

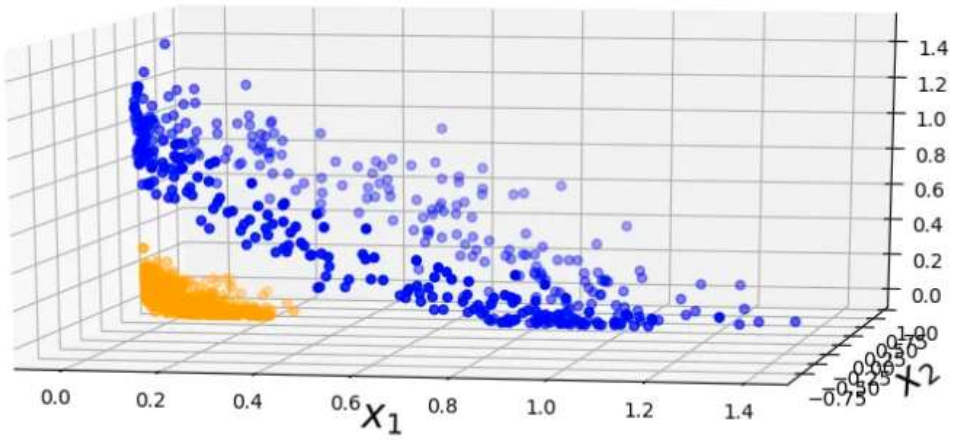
Original data, which cannot be linearly separated



Original data, which cannot be linearly separated

Apply second degree polynomial mapping

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2}\, x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$



The data becomes linearly separable (by a 2-d plane) in 3-dimensions

Kernel Definition

- A function that takes as its inputs vectors in the original space and returns the dot product of the vectors in the feature space is called a *kernel function*
- More formally, if we have data $\mathbf{x}, \mathbf{z} \in X$ and a map $\phi : X \to \Re^N$ then

$$k(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

is a kernel function

- Note: k(x, z) is a kernel function
- Φ is a mapping function – converts data from input space to feature space
- x, z are data points
- So involves computational cost of

1. Mapping every data point to feature space and
2. then computing their dot product as per above kernel function definition

- So kernel trick is employed

What are Kernel Functions and its different types?

- The kernel function is a function which can be represented the dot product of the mapping function (kernel method)

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

- Kernel function reduces the complexity of finding the mapping function.
- So, the kernel function defines the inner product in the transformed space.
- Different types of kernel functions
- Polynomial kernel
- Gaussian kernel
- Radial basis function (RBF) kernel
- Sigmoid kernel
- Note: one can define their own kernel functions based on the condition that kernel function must be symmetric and satisfies the Mercer theorem

Common Types of Kernels used in SVM

1. Linear Kernel

- We have two vectors x1 and x2.
- The linear kernel is defined by the dot product of these two vectors:
- K(x1, x2) = x1 * x2

2. Polynomial Kernel

- A polynomial kernel is defined by the following equation: $K(x1, x2) = (x1 * x2 + c)^d$
- $K(x1, x2) = (x1 * x2)^d$
- where d is the degree of the polynomial, d=2 is quadratic kernel and x1 and x2 are vectors. Constant c =0.

3. Gaussian Kernel also known as the radial basis function (RBF) kernel

$$K(x, y) = e^{-\left(\frac{||x-y||^2}{2\sigma^2}\right)}$$

x and y are the input feature vectors, sigma is a parameter that controls the width of the Gaussian function, and ||x - y||^2 is the squared Euclidean distance between the input vectors.

Problem statement 7: Apply kernel function and show that applying the kernel function is equivalent to applying the mapping function

Kernel trick

- Performing kernel trick is much easier compared to mapping functions
- Helps in converting input space to feature space with least computations
- Given two points (1, 2) and (3, 4), apply polynomial kernel function $k(x, y) = (x^T. y)^2$ and show that it is equivalent to mapping function $\phi = (x^2, y^2, \sqrt{2}xy)$
- Polynomial kernel here is quadratic

Problem statement 8: Apply kernel functions to the given data points

Apply kernel functions to the given data points

1. Apply linear and polynomial kernel function to the data points (1

   2) and (2, 3)

2. Convert the data point (2,3) to 3-D data space by applying quadratic mapping function

3. Apply RBF to the data points (1, 2) and (2, 3). Assume **σ = 1**

$$K(x, y) = e^{-(\frac{||x-y||^2}{2\sigma^2})}$$

Kernel trick

- Solution of the dual problem gives us:

$$\hat{\mathbf{w}} = \sum_{i=1}^{n} \hat{\alpha}_i y_i \mathbf{x}_i$$

- The decision boundary:

$$\hat{\mathbf{w}}^T \mathbf{x} + w_0 = \sum_{i \in SV} \hat{\alpha}_i y_i (\mathbf{x}_i^T \mathbf{x}) + w_0$$

- The decision:

$$\hat{y} = \text{sign}\left[\sum_{i \in SV} \hat{\alpha}_i y_i (\mathbf{x}_i^T \mathbf{x}) + w_0 \right]$$

- Mapping to a feature space, we have the decision:

$$\hat{y} = \text{sign}\left[\sum_{i \in SV} \hat{\alpha}_i y_i (\phi(\mathbf{x}) \cdot \phi(\mathbf{x}_i)) + w_0 \right]$$

In the last equation, we replace the dot product of the transformed vectors with the kernel function

Kernel trick

- There can be many transformations that allow the data to be linearly separated in higher dimensions
- But not all of these functions are actually kernels.
- The kernel function has a special property that makes it particularly useful in training support vector models
- The use of this property in optimizing non-linear support vector classifiers is often called the kernel trick.

Kernel trick

- In real applications, there might be many features in the data
- Applying transformations that involve many polynomial combinations of these features will lead to extremely high and impractical computational costs.
- The kernel trick provides a solution to this problem.
- The "trick" is that kernel methods represent the data only through a set of pairwise similarity comparisons between the original data observations x (with the original coordinates in the lower dimensional space),
- instead of explicitly applying the transformations φ(x) and representing the data by these transformed coordinates in the higher dimensional feature space.

Kernel trick

- In kernel methods, the data set X is represented by an *n x n* kernel matrix of pairwise similarity comparisons
- The entries (i, j) are defined by the kernel function: k(xi, xj).
- This kernel function has a special mathematical property.
- The kernel function acts as a modified dot product.
- kernel function accepts inputs in the original lower dimensional space and
- returns the dot product of the transformed vectors in the higher dimensional space.

Kernel trick

- How the kernel function is equal to the dot product of the transformed vectors
- Each coordinate of the transformed vector φ(x) is some function of the coordinates in the corresponding lower dimensional vector x.

- Ex: the kernel trick for the 2nd-degree polynomial
- The transformed vectors have coordinates that are functions of the two components x1 and x2.
- so the dot product will only involve components x1 and x2
- The kernel function will also take inputs x1, x2 and return a real number
- The dot product always returns a real number

Kernel trick

- The benefit of the kernel trick
- The objective function we are optimizing to fit the higher dimensional decision boundary only includes the dot product of the transformed feature vectors.
- Therefore, we can just substitute these dot product terms with the kernel function, and we do not use $\varphi(x)$
- Kernel Trick is essentially the use of kernel functions, which enables to operate in a high-dimensional, implicit feature space without ever computing the coordinates of the data in that space.
- This operation is often computationally cheaper than the explicit computation of the coordinates.

Kernel Trick

Show that applying the kernel function is equivalent to applying the mapping function

- Instead of transforming

$(q1, p1)$ into $(q_1^2, \sqrt{2}q_1p_1, p_1^2)$ and

$(q2, p2)$ into $(q_2^2, \sqrt{2}q_2p_2, p_2^2)$ and

- then computing the dot-product of
- $(q_1^2, \sqrt{2}q_1p_1, p_1^2) \cdot (q_2^2, \sqrt{2}q_2p_2, p_2^2) = (q_1^2q_2^2 + 2q_1q_2p_1p_2 + p_1^2q_2^2)$
- We could find the dot-product of
- $(q1, p1) \cdot (q2, p2) = (q1q2+p1p2)$ and
- then square it to get exactly the same result $(q_1^2q_2^2 + 2q_1q_2p_1p_2 + p_1^2q_2^2)$

Kernel trick

Show that applying the kernel function is equivalent to applying the mapping function

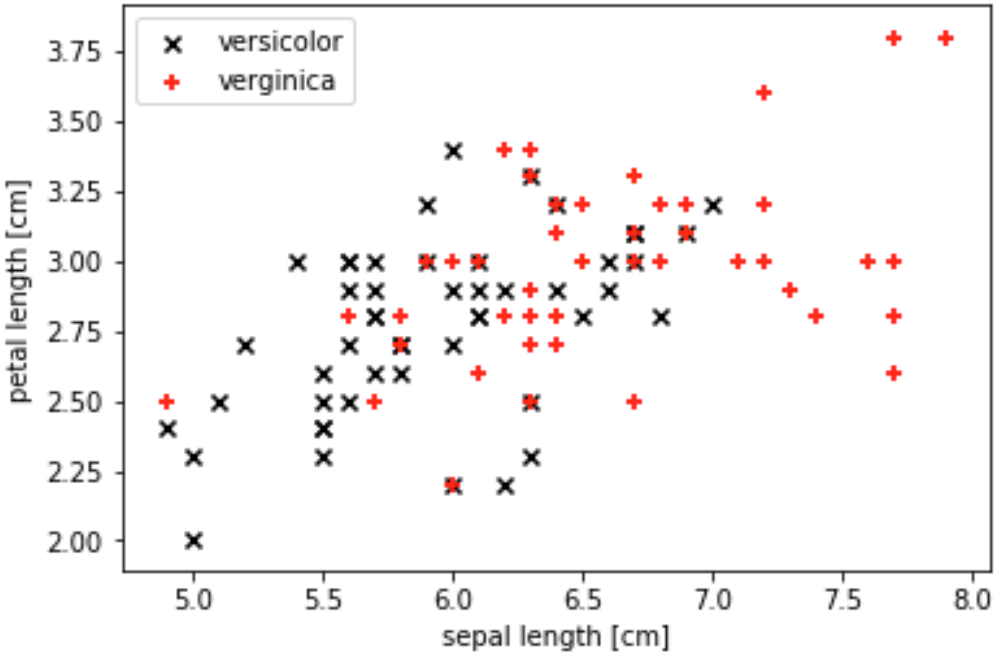
$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

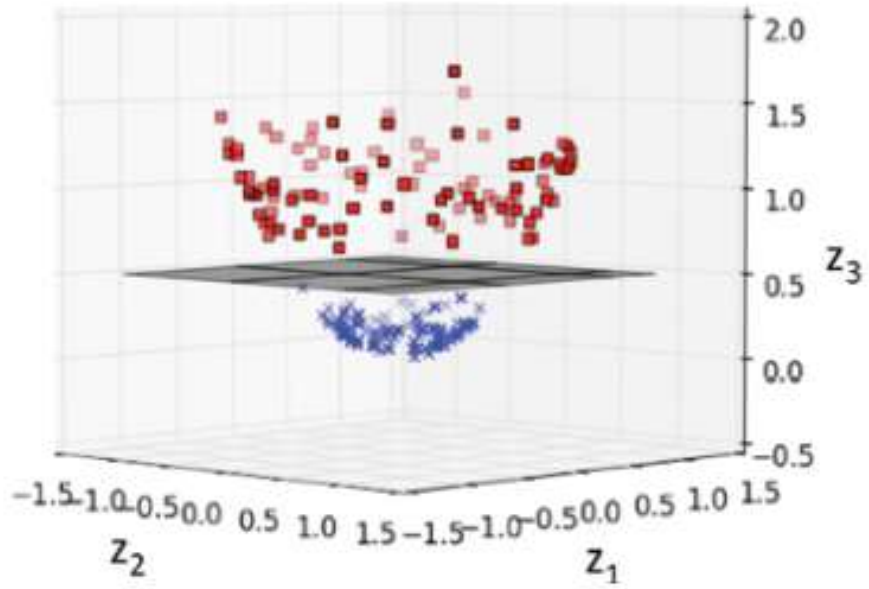
$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b} \right)^2$$

On the left-hand side, we have the dot product of the transformed feature vectors, which is equal to our 2nd-degree polynomial kernel function

The kernel function here is the polynomial kernel k(a,b) = (a^T * b)²

The Kernel Concept





- Fig 1: The IRIS data set which is linearly inseparable. Fig 2: Non-linear data represented using 3 dimensions
- The above data set represented by adding another dimension.
- Data in two-dimension gets transformed when moved to 3-dimension. The data set becomes linearly separable after applying appropriate kernel function
- A hyperplane can be found which separates them clearly.

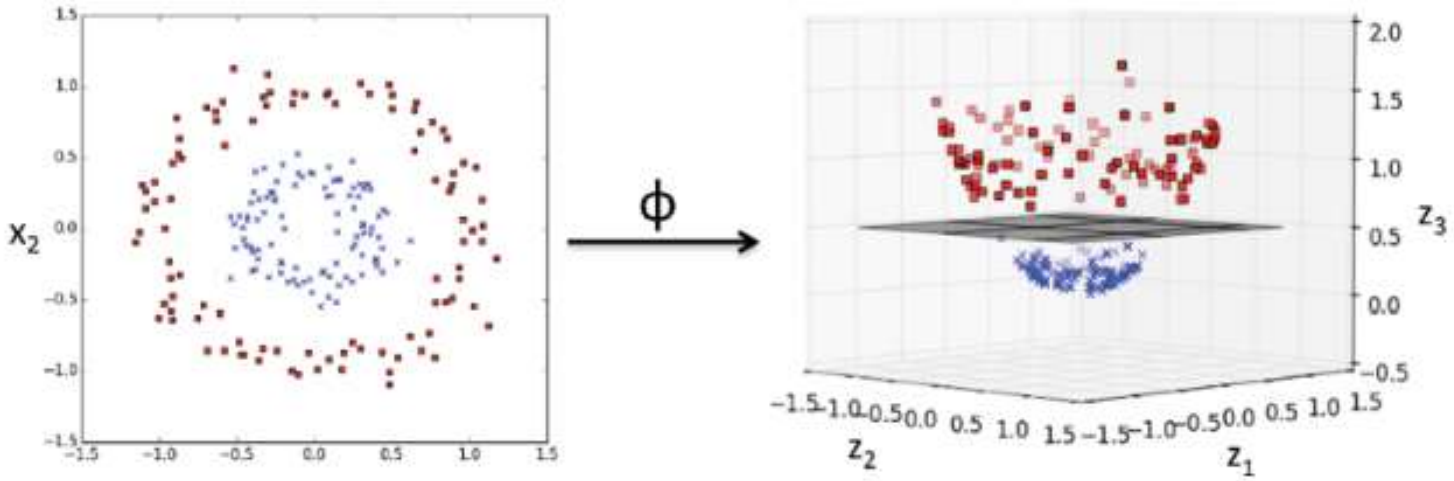
The Kernel Concept

- The kernel function here is the polynomial kernel $k(a,b) = (a^T * b)^2$
- The benefit of the kernel trick is that the objective function we are optimizing to fit the higher dimensional decision boundary only includes the dot product of the transformed feature vectors.
- Therefore, we can just substitute these dot product terms with the kernel function, and we do not even use $\varphi(x)$

Kernel Methods

- Kernel Methods represent the techniques that are used to deal with linearly inseparable data or non-linear data set shown
- The idea is to create nonlinear combinations of the original features to project them onto a higher-dimensional space via a mapping function, where the data becomes linearly separable.

Kernel Methods



Dataset projected into 3-dimension feature space
The two-dimensional dataset (X1, X2) is projected into a new three-dimensional feature space (Z1, Z2, Z3) where the classes become separable

Kernel Methods

- By using the above method, the training data can be transformed into a higher-dimensional feature space via a mapping function
- A linear SVM model can be trained to classify the data in this new feature space.
- The new data can then be transformed using the mapping function and fed into the model for classification.
- However, this method is computationally very expensive.
- Solution?

Kernel Tricks

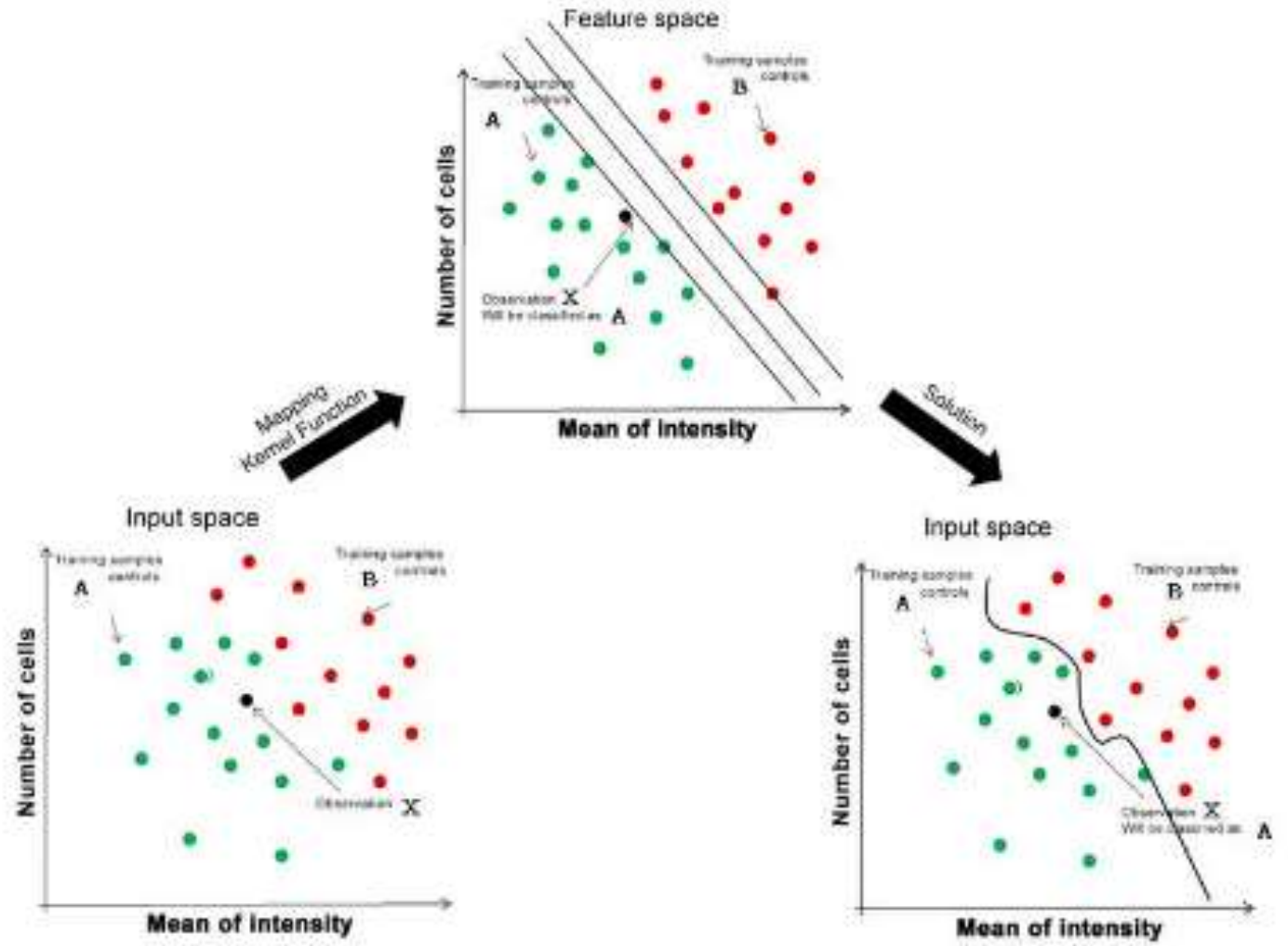
- The solution is to apply some trick
- Which can avoid the need to explicitly map the input data to high-dimension feature space in order to train linear learning algorithms to learn a nonlinear function or decision boundary
- This is called a kernel trick
- To deal with the above challenge of computing (expensive) the data coordinates (new features) using mapping function to transform to higher-dimensional space, the kernel trick is applied.
- Kernel trick is to convert dot product of support vectors to the dot product of mapping function.

Kernel Tricks

- SVM uses kernel trick, as can be seen on the left, to transform the data to a different dimension to get the desired result.
- The result is then brought back to 2-dimensional space in the form of classified labels to be used in real life.
- The analysis of the large amount of data generated in Cell-based High-Content Screening (HCS) experiments
- Here SVM was used to find the line (or plane) that best separates groups of genes as known and unknown samples i.e. positive (A) and negative (B) samples.
- The features were number of cells in each sample and the intensity/density of the cells in the samples

Kernel Trick

Transformation of data from Input Space to a Higher Dimension Feature Space for a linear separation, followed by a transformation back to input space where the classification boundary looks non-linear.



Support Vector Machines (Kernels)- Summary

- The SVM algorithm is implemented in practice using a kernel
- The learning of the hyperplane in linear SVM is done by transforming the problem using linear algebra
- A powerful insight is that the linear SVM can be rephrased using the inner product of any two given observations, rather than the observations themselves
- The inner product between two vectors is the sum of the multiplication of each pair of input values.
- Ex: the inner product of the vectors [2, 3] and [5, 6] is 2 * 5 + 3 * 6 = 28.
- The equation for making a prediction for a new input using the dot product between the input (x) and each support vector (xi) is calculated as follows:

$$f(x) = B0 + \sum_{i=1}^{n} (a_i \times (x \times x_i))$$

- This is an equation that involves calculating the inner products of a new input vector (x) with all support vectors in training data.
- The coefficients B0 and a_i (for each input) must be estimated from the training data by the learning algorithm.

Support Vector Machines (Kernels) - Summary

- Linear Kernel SVM
- The dot-product is called the kernel and can be re-written as:
- $K(x, xi) = \Sigma\ (x * xi)$
- The kernel defines the similarity or a distance measure between new data and the support vectors.
- The dot product is the similarity measure used for linear SVM or a linear kernel because the distance is a linear combination of the inputs.
- Kernel Trick
- Other kernels can be used that transform the input space into higher dimensions such as a Polynomial Kernel and a Radial Kernel.
- This is called the Kernel Trick.
- It is desirable to use more complex kernels as it allows lines to separate the classes that are curved or even more complex.
- This in turn can lead to more accurate classifiers

Soft Margin

Hard Margin & Soft Margin

- Margin can be sub-divided:
- 1. Hard Margin – If the training data is linearly separable, we can select two parallel hyperplanes that separate the two classes of data, so that the distance between them is as large as possible.
- In order to find the maximal margin, we need to maximize the margin between the data points and the hyperplane.
- 2. Soft Margin – As most of the real-world data are not fully linearly separable, we will allow some margin violation to occur which is called soft margin classification.
- It is better to have a large margin, even though some constraints are violated.
- Margin violation means choosing a hyperplane, which can allow some data points to stay on either the incorrect side of the hyperplane and between the margin and correct side of the hyperplane.

Hard Margin & Soft Margin

